algorithms

MDPI

Systematic Review

Artificial Intelligence in Software Testing: A Systematic Review of a Decade of Evolution and Taxonomy

Alex Escalante-Viteri * and David Mauricio

Faculty of Systems and Informatics Engineering, Universidad Nacional Mayor de San Marcos, Lima 15081, Peru; dmauricios@unmsm.edu.pe

* Correspondence: alex.escalante@unmsm.edu.pe

Abstract

Software testing is fundamental to ensuring the quality, reliability, and security of software systems. Over the past decade, artificial intelligence (AI) algorithms have been increasingly applied to automate testing processes, predict and detect defects, and optimize evaluation strategies. This systematic review examines studies published between 2014 and 2024, focusing on the taxonomy and evolution of algorithms across problems, variables, and metrics in software testing. A taxonomy of testing problems is proposed by categorizing issues identified in the literature and mapping the AI algorithms applied to them. In parallel, the review analyzes the input variables and evaluation metrics used by these algorithms, organizing them into established categories and exploring their evolution over time. The findings reveal three complementary trajectories: (1) the evolution of problem categories, from defect prediction toward automation, collaboration, and evaluation; (2) the evolution of input variables, highlighting the increasing importance of semantic, dynamic, and interface-driven data sources beyond structural metrics; and (3) the evolution of evaluation metrics, from classical performance indicators to advanced, testing-specific, and coverage-oriented measures. Finally, the study integrates these dimensions, showing how interdependencies among problems, variables, and metrics have shaped the maturity of AI in software testing. This review contributes a novel taxonomy of problems, a synthesis of variables and metrics, and a future research agenda emphasizing scalability, interpretability, and industrial adoption.

Keywords: artificial intelligence; software testing; systematic review; defect prediction; software metrics; test automation; taxonomy; explainable AI

check for updates

Academic Editor: Alexander E.I. Brownlee

Received: 9 October 2025
Revised: 1 November 2025
Accepted: 4 November 2025
Published: 14 November 2025

Citation: Escalante-Viteri, A.; Mauricio, D. Artificial Intelligence in Software Testing: A Systematic Review of a Decade of Evolution and Taxonomy. *Algorithms* **2025**, *18*, 717. https://doi.org/10.3390/a18110717

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In the digital era, software is a fundamental engine driving modern technology. Its relevance is manifested in its ability to transform data into useful information, to automate processes, and to foster efficiency and innovation across various industrial sectors. As the core of digital transformation, software not only facilitates digitization but also creates unprecedented business opportunities. According to a study by McKinsey & Company, firms that adopt advanced digital technologies, including software, can achieve significantly increased productivity and competitiveness [1]. Moreover, software plays a key role in developing new applications that are transforming sectors such as healthcare, education, and transportation, thereby reshaping the economic and social landscape [2]. The software industry also contributes significantly to the global economy by improving productivity and efficiency across other sectors [3]. In terms of security, it protects personal

and corporate data against cyber threats [4] and has revolutionized teaching and learning methods through the development of interactive and accessible platforms that enhance educational effectiveness [5].

Software testing (ST) is a critical phase in the development cycle that ensures the quality and functionality of the final product [6]. Since 57% of the world's population uses internet-connected applications, it is imperative to develop secure, high-quality software to avoid the risk of significant harm, including major financial losses [7]. The inherent complexity and defects in software require that approximately 50% of development time be devoted to testing, which is essential to ensure the delivery of high-quality products [8].

The introduction of artificial intelligence (AI) algorithms is revolutionizing ST, making it more intelligent, efficient, and accurate. These algorithms enhance testing processes by reducing the time and costs involved [9]. Techniques such as machine learning (ML) allow for analysis of source code or expected application behavior, to enable more exhaustive tests to be generated and potential errors identified. They are also used in data mining and clustering to prioritize critical areas of the code and to enable automatic test case generation (TCG) [10–12]. Moreover, genetic and search-based algorithms are employed in automated interface validation and the generation of software defect prediction (SDP) models to identify parts of the code that are more prone to failure based on factors such as code complexity and defect history [13–16]. This enables testing efforts to focus on critical areas, thereby increasing efficiency and reducing the test time.

In recent years, advances in AI algorithms have significantly transformed the domain of ST, with notable impacts across various key areas. For example, the authors of [17] applied deep learning (DL) techniques using object detection algorithms such as EfficientDet and Detection Transformer, along with text generation models like GPT-2 and T5, achieving outstanding accuracy rates of 93.82% and 98.08% in TCG. In another study [18], researchers used ML methods for software defect detection, achieving an impressive accuracy of 98%. Similarly, the authors of [19] explored the use of neural networks (NNs) with natural language processing (NLP) models for e-commerce applications, reporting excellent results of 98.6% and 98.8% in correct test case generation. In [9], NNs were applied to calculate the failure-proneness score, giving high-level metrics that supported their effectiveness. Finally, the study in [20] highlighted the potential of DL in software fault prediction, with a confidence level of 95%.

The growing number of studies on the use of AI in ST has prompted researchers to conduct systematic literature reviews. The authors of [21] highlighted ML-based defect prediction methods, although they noted a lack of practical applications in industrial contexts. In [22], the increasing use of AI was confirmed, whereas in [23], NLP-based approaches were investigated for requirements analysis and TCG, with challenges such as the generalization of algorithms across domains being identified. The researchers in [24] classified ML methods applied to testing, while those in [25] observed a decline in traditional methods in favor of innovations such as automatic program repair. In [26], the current lack of theoretical knowledge in anticipatory systems testing was emphasized. In [27], the development of generalized metamorphic rules for testing AI-based applications was promoted. Finally, the authors of [28,29] analyzed improvements in test case prioritization and generation using ML techniques and highlighted the urgent need for further research to align academic work with industrial demand.

AI algorithms can have a significant impact on ST by improving the testing time, accuracy, and overall quality. It is therefore essential to address the question of how AI algorithms have evolved in ST; this will help to highlight the advances in these algorithms and their growing importance in ST, since AI enables the automation and optimization of

tests, reduces human error and development time, and facilitates the early detection of complex defects.

The purpose of this study is to analyze and explore the evolution in the use of AI algorithms for ST from 2014 to 2024, with the aim of helping quality engineers and software developers identify the relevant AI algorithms and their applications in ST, while also supporting researchers in the development of new approaches. To achieve this, a systematic literature review will be conducted on AI algorithms in ST.

This paper makes the following contributions to the field of AI-based software testing:

(1) A taxonomy of problems in software testing, proposed by the authors by creating categories according to the issues identified in the reviewed literature.
(2) A systematization of input variables used to train AI models, organized into thematic categories, with special emphasis on structural source code metrics and complexity/quality metrics as drivers of algorithmic focus.
(3) A synthesis of performance metrics applied to assess the effectiveness and robustness of AI models, distinguishing between classical performance indicators and advanced classification measures.
(4) An integrative and evolutionary perspective that highlights the interplay between problems, input variables, and performance metrics, and traces the maturation and diversification of AI in software testing.
(5) A future research agenda that outlines open challenges related to scalability, interpretability, and industrial adoption, while drawing attention to the role of hybrid and explainable AI approaches.

This article is organized into six sections. Section 2 reviews ST. Section 3 presents a systematic literature review of the use of AI algorithms in ST, while their evolution, including variables and metrics, is described in Section 4 Finally, a discussion and some conclusions are presented in Sections 5 and 6, respectively.

2. Software Testing (ST)

2.1. Concept and Advantages

ST originated in the 1950s, when software development began to be consolidated as a structured, systematic activity. In its early days, ST was considered an extension of debugging, with a primary focus on identifying and correcting code errors. During the 1950s and 1960s, testing was mostly ad hoc and informal and was done with the aim of correcting failures after their detection during execution. However, in the 1970s, a more systematic approach emerged with the introduction of formal testing techniques, which contributed to distinguishing ST from debugging. Glenford J. Myers was one of the pioneers in establishing ST as an independent discipline through his seminal work The Art of Software Testing [30].

ST is a systematic process carried out to evaluate and verify whether a program or system meets specified requirements and functions as intended. It involves the controlled execution of applications with the goal of detecting errors. According to the ISO/IEC/IEEE 29119 Software Testing Standard [31]. ST is defined as "a process of analyzing a component or system to detect the differences between existing and required conditions (i.e., defects) and to evaluate the characteristics of the component or system". Accordingly, ST has several objectives: verifying functionality, identifying defects, validating user requirement compliance, and improving the overall quality of the final product [32].

The systematic application of ST ensures that the final product meets the required quality standards. By detecting and correcting defects prior to release, software reliability and functionality are enhanced. In [33], it was asserted that systematic testing is essential to ensure proper performance under all intended scenarios. Moreover, ST contributes to

long-term cost reductions, as the cost of correcting a defect increases exponentially the later it is discovered in the software life cycle [34]. In addition, security testing plays a key role in preventing fraud and protecting sensitive information, making it an essential component of secure software development [35]. Finally, ST delivers the promised value to the user [36] and facilitates future maintenance, provided that the software is free from significant defects [37].

2.2. Forms of Software Testing

For a better understanding, software testing can be classified into four main dimensions: testing level, testing type, testing approach, and degree of automation, as described below.

By testing level:

- Unit Testing (UTE): This focuses on validating small units of code, such as individual functions or methods, as these are the closest to the source code and the fastest to execute [38].
- Integration Testing (INT): This evaluates the interaction between different modules or components to ensure that they work together correctly [39].
- System Testing (End-to-End): The aim of this is to simulates complete system usage to verify that all components function properly from the user's perspective [40].
- Acceptance Testing (ACT): This is conducted to validate that the software meets the client's requirements or acceptance criteria before release [41].
- Stress and Load Testing (SLT): In this approach, the system's behavior is analyzed under extreme or high-demand conditions.

By test type:

- Functional Testing (FUT): This ensures that the software fulfills the specified functionalities [42].
- Non-functional Testing (NFT): This is conducted to evaluate attributes that are related to performance and external quality rather than directly to internal functionality. It includes:
- Performance Testing (PET): This analyzes response times, load handling, and capacity under different conditions.
- Security Testing (SET): This is done to verify protection against attacks or unauthorized access.
- Usability Testing (UST): This assesses the user experience. Although usually conducted manually, some aspects such as accessibility may be partially automated [43].

By testing approach:

- Test-Driven Development (TDD): Tests are written before the code, guiding the development process. The input data and expected results are stored externally to support repeated execution [44].
- Behavior-Driven Development (BDD): Tests are formulated in natural language and aligned with business requirements [44].
- Keyword-Driven Testing (KDT): Predefined keywords representing actions are used, which separate the test logic from the code and allow non-programmers to create tests [45].

By degree of automation:

- Automated Testing (AUT): This involves the use of tools such as Selenium or Cypress to interact with the graphical user interface [46], JUnit for unit testing in Java, or Appium in mobile environments for Android/iOS. Backend or API tests are typically conducted using Postman, REST-assured, or SoapUI.

- Fully Automated Testing (FAT): The entire testing cycle (execution and reporting) is carried out without human intervention [47].
- Semi-Automated Testing (SAT): In this approach, part of the process is automated, but human involvement is required in certain phases, such as result analysis or environment setup [47].

2.3. Standards

ST is governed by a set of internationally recognized standards that define best practices, processes, and requirements to ensure quality and consistency throughout the testing life cycle. The primary framework is established by the ISO/IEC/IEEE 29119:2013 standard [31], which provides a comprehensive foundation for software testing concepts, processes, documentation, and evaluation techniques. Complementary standards, such as ISO/IEC 25010 (Software Product Quality Model), IEEE 1028 (Software Reviews and Audits), and ISO/IEC/IEEE 12207 (Software Life Cycle Processes), extend this framework by addressing aspects of product quality, review procedures, and integration of testing activities into the broader software development process. Together, these standards ensure alignment with international software quality assurance practices and provide a structured basis for the systematic application of ST.

2.4. Aspects of Software Testing

There are several aspects of ST that contribute to ensuring the quality of the final product. These are illustrated in Figure 1 and described below:

- Techniques and Strategies: These refer to the methods and approaches used to design, execute, and optimize software tests, such as test case design, automation, and risk-based testing. The aim of these is to maximize the efficiency and coverage of the testing process [48].
- Tools and Technology: These involve the collection of systems, platforms, and tools employed to support testing activities, from test case management to automation and performance analysis, thereby facilitating integration within modern development environments such as CI/CD [48].
- Software Quality: This encompasses a set of attributes such as functionality, maintainability, performance, and security, which determine the level of software excellence, supported by metrics and evaluation techniques throughout the testing cycle [49].
- Organization: This refers to the planning and management of the testing process, including role assignments, team integration, and the adoption of agile or DevOps methodologies, to ensure alignment with project goals [50].
- AI Algorithms in ST: The use of AI involves the application of techniques such as ML, data mining, and optimization to enhance the efficiency, effectiveness, and coverage of the testing process. These tools enable intelligent TCG, defect prediction, critical area prioritization, and automated result analysis, thereby significantly reducing the manual effort required [51].
- Innovation and Research: These include the exploration of advanced trends such as the use of AI, explainability in testing, and validation of autonomous systems, which contribute to the development of new techniques and approaches to address challenges in ST 52.
- Future Trends: These refer to emerging and high-potential areas such as IoT system validation, testing in the metaverse, immersive systems, and testing of ML models, which reflect technological advances and new demands in software development [52].

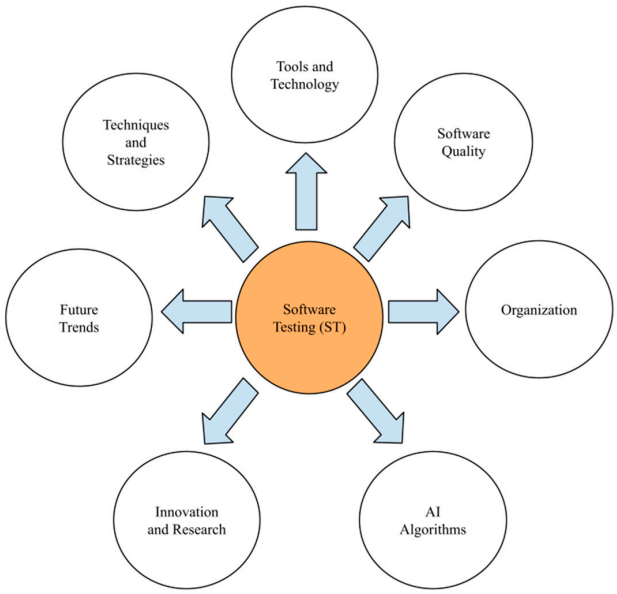


Figure 1. Aspects of software testing.

3. Systematic Literature Review on AI Algorithm in Software Testing

In view of the relevance of the use of AI in ST and its impact on software quality, it is essential to conduct a comprehensive literature review to identify and analyze recent advancements and contributions in this field. To achieve this, it is necessary to adopt a structured methodology that allows for the efficient organization of information.

3.1. Methodology

This systematic literature review was conducted following the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA 2020) guidelines. The PRISMA 2020 checklist and flow diagram have been included as Supplementary Materials to ensure methodological transparency and reproducibility.

The methodology for this state-of-the-art study is based on a guideline that was initially proposed in [53], and which has been adapted for systematic literature reviews in software engineering. This approach has been widely applied in related research, including the use of model-based ST tools [54], general studies of ST [55], investigations of software quality [56], and software defect prediction using AI techniques [57]. The review process consists of four stages: planning, execution, results, and analysis.

3.2. Planning

To explore the evolution of AI algorithms in ST, the following research questions were formulated:

RQ1: Which AI algorithms have been used in ST, and for what purposes?

RQ2: Which variables are used by AI algorithms in ST?

RQ3: Which metrics are used to evaluate the results of AI algorithms in ST?

To answer these questions, a journal article search strategy was developed based on a specific search string, including Boolean operators and applied filters, as detailed in Table 1, ensuring transparency and reproducibility according to PRISMA 2020 guidelines. The selection of keywords reflected the relevant aspects and context of the study, and the search was carried out using the Scopus and Web of Science (WoS) databases. These databases were chosen due to their extensive peer-reviewed coverage, continuous inclusion of new journals, frequent updates, and relevance in terms of providing up-to-date impact metrics, stable citation structures, and interoperability with bibliometric tools, which are crucial for

automated data curation and large-scale analysis. Inclusion and exclusion criteria were established to filter and select relevant studies, as specified in Table 2.

Table 1. Search strings used with Database.

Database	Search String
Scopus	TITLE-ABS-KEY ((“method” OR “procedure” OR “guide”) AND (“software test” OR “software testing”) AND (“artificial intelligence” OR “machine learning” OR “deep learning” OR “generative AI” OR “genAI”))
WoS	(“method” OR “procedure” OR “guide”) AND (“software test” OR “software testing”) AND (“artificial intelligence” OR “machine learning” OR “deep learning” OR “generative AI” OR “genAI”) (Topic)

Scopus and Web of Science (WoS) were selected because they index journals from IEEE Xplore and the ACM Digital Library, ensuring a representative and up-to-date dataset for this systematic review. The quartile distribution of the reviewed journals is detailed in the *Results* section (Section 3.4), which includes IEEE Xplore–indexed sources such as *IEEE Access*. Additionally, during the filtering stage based on abstracts and keywords, several journals from the ACM Digital Library were identified as duplicates. This overlap confirms that the relevant IEEE and ACM publications had already been captured through the Scopus and WoS searches.

Table 2. Inclusion and exclusion criteria.

Inclusion Criteria	Exclusion Criteria
Peer-reviewed journal articles Studies published between 2014 and 2024 Articles written in English Research explicitly applying AI algorithms to software testing Studies reporting experimental results or comparative analyses	Non-peer-reviewed publications (e.g., theses, dissertations, technical reports, white papers) Studies not written in English Duplicated records from multiple databases Articles published before 2014 or after 2024 Papers not related to AI-based software testing

We acknowledge that the rapid impact of emerging technologies in Software Engineering may reshape any existing taxonomy and its evolution over time. Some of these developments are often first introduced at leading international conferences (e.g., ICSE, FSE, ISSTA), reflecting the continuous adaptation of the field to new requirements—particularly those driven by advances in Artificial Intelligence models. While this dynamism is inevitable, the taxonomy proposed in this study remains valuable as a foundational scientific framework that can guide future refinements and inspire further research addressing contemporary challenges in software testing. Furthermore, future extensions of this systematic review may incorporate peer-reviewed conference proceedings from these venues to broaden the scope and capture cutting-edge contributions that often precede journal publications.

The final search string was iteratively refined to balance inclusiveness and precision, ensuring the retrieval of relevant studies without excessive noise. During the filtering process, when searching for software testing methods, the databases consistently returned studies addressing software defect prediction, test case prioritization, fuzzing, and other key topics that directly contributed to defining the proposed taxonomy.

This empirical verification supports that, although more specialized keywords could have been included, the applied search string effectively captured the main families of studies relevant to the research questions. In addition to general methodological terms (“method,” “procedure,” “guide”), domain-specific terminology was already embedded within the retrieved dataset through metadata and indexing structures in Scopus and WoS.

Furthermore, the validity of the search strategy was implicitly supported through the PRISMA-based screening and deduplication process, which acted as a quality control mechanism comparable to a “gold standard” verification. This ensured that the taxonomy and trend analysis reflected a comprehensive and representative overview of AI-driven software testing research.

3.3. Execution

According to the previously defined planning strategy, the initial search yielded 1985 articles from Scopus and 3447 from WoS, resulting in a total of 5432 articles. Using a filtering tool based on predefined exclusion criteria, this number was significantly reduced by eliminating 4217 articles, leaving a total of 1215.

Subsequently, 183 duplicate articles were removed (182 from WoS and one from Scopus). In addition, three retracted articles were excluded, including two from WoS and one from Scopus. As a result, 1029 articles remained for further detailed screening using additional filters.

The filters that were applied were as follows:

- Title: 676 articles were excluded (173 from Scopus and 503 from WoS)
- Abstract and Keywords: 246 articles were removed (134 from Scopus and 112 from WoS)
- Introduction and Conclusion: Nine articles were excluded (seven from Scopus and two from WoS)
- Full Document Review: 10 articles were rejected (eight from Scopus and two from WoS)

This process excluded 941 articles, leaving a total of 88 for in-depth review. Of these, 22 were excluded as they did not directly address the proposed research questions, resulting in 66 articles which were selected as relevant in answering the research questions.

The literature search covered 2014–2024 and was last updated on 30 September 2024 across Scopus and Web of Science; search strings were adapted per database (Table 1). The inclusion and exclusion criteria used to filter studies are detailed in Figure 2, following PRISMA 2020 recommendations [58] based on selection parameters in Table 2.

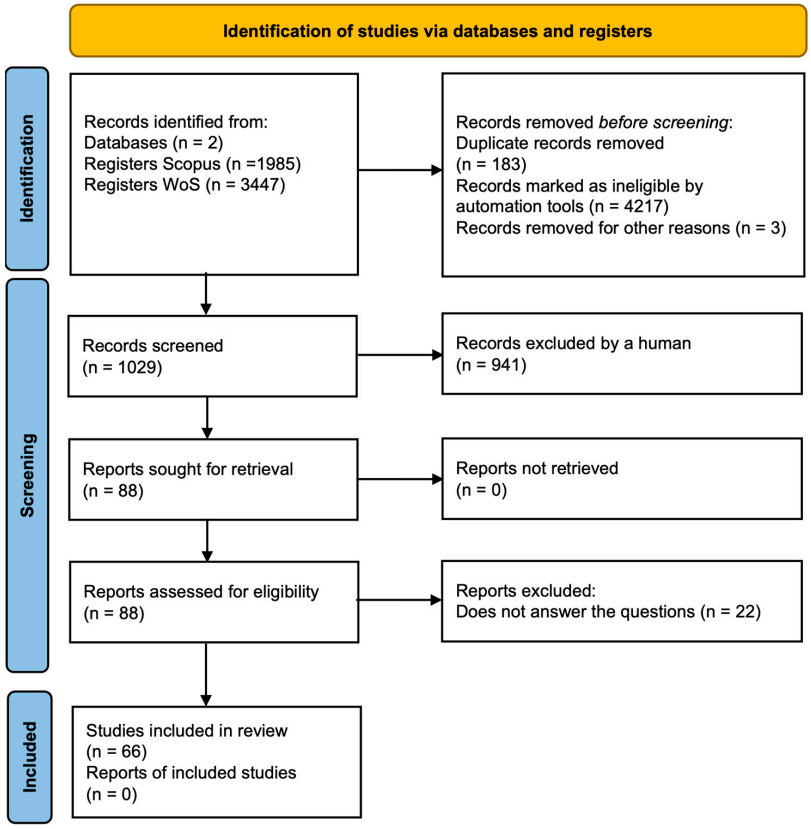


Figure 2. PRISMA 2020 flow diagram of the systematic review process. Adapted from Page et al. (2021) [58], PRISMA 2020 guideline.

Data Screening and Extraction Process

The selection and data extraction processes were carried out by two independent reviewers (A.E., D.M.) who applied predefined inclusion and exclusion criteria across four sequential stages: title screening, abstract and keyword review, introduction and conclusion assessment, and full-text analysis. Each reviewer performed the screening independently, and any discrepancies were resolved through discussion and consensus. The process was supported using Microsoft Excel to ensure traceability and consistency across all stages. For each selected study, information was extracted regarding the publication year, algorithm type, testing problem category, input variables, evaluation metrics, and datasets used. The extracted information was cross-checked with the original articles to ensure completeness and accuracy, and the consolidated dataset served as the basis for the analytical synthesis presented in the following sections. The overall workflow is summarized in Figure 2, following the PRISMA 2020 flow diagram.

To further ensure methodological rigor and minimize bias, the screening and data extraction stages were conducted independently by both reviewers, with all decisions cross-verified and reconciled through consensus. Although a formal inter-rater reliability coefficient (e.g., Cohen's κ) was not computed, the dual-review approach followed established SLR practices in software engineering, ensuring transparency, traceability, and reproducibility throughout the process.

In terms of study quality, all included papers were peer-reviewed journal publications indexed in Scopus and WoS, guaranteeing a baseline of methodological soundness. As illustrated in the results in Section 3.4, 54.5% of the selected studies were published in Q1 journals, reflecting the high scientific quality and credibility of the dataset. Consequently, an additional numerical quality scoring was deemed unnecessary. Nevertheless, we recognize that future reviews could be strengthened by incorporating a formal quality assessment checklist (e.g., Kitchenham & Charters, 2007 [53]) and quantitative reliability metrics to further enhance objectivity and consistency.

To strengthen transparency and reproducibility, all key artifacts from the systematic review have been made publicly available in the supplementary repository (https://github.com/escalasoft/ai-software-testing-review-data (accessed on 31 October 2025). The repository includes:

(1) filtering_articles_marked.xlsx, documenting the screening stages across title, abstract/keywords, and introduction/conclusion, along with complementary filters such as duplicates, retracted papers, and studies not responding to the research question.
(2) raw_data_extracted.xlsx, containing the raw data extracted from each selected study, including problem codes (e.g., SDP, TCM, ATE), dataset identifiers, algorithm names, number of instances, and evaluation metrics (e.g., Accuracy, Precision, Recall, F1-score, ROC-AUC);
(3) coding_book_taxonomy.xlsx, defining the operational rules applied to classify studies into taxonomy categories.
(4) PRISMA_2020_Checklist.docx, presenting the full checklist followed during the review.

Additional details on algorithms, variables, and metrics are included in the Appendices B–D. Together, these materials ensure full traceability and compliance with PRISMA 2020 guidelines.

3.4. Results

3.4.1. Potentially Eligible and Selected Articles

Our systematic literature review resulted in the selection of 66 articles that met the established criteria and were relevant to addressing the research questions. These articles

are denoted using references in the format [n]. The complete list of selected studies is provided in Appendix A. Table 3 presents a summary of the potentially eligible articles and those ultimately selected after the review process.

Table 3. Potential and selected articles.

Source	# Potentially Eligible Studies	# Selected Articles	Selected Articles
Scopus	79	59	[17–20,59–113]
Web of Science	9	7	[114–120]
Total	88	66	[17–20,59–120]

3.4.2. Publication Trends

Figure 3 reveals a trend towards greater numbers of publications on AI algorithms in ST over the past decade. From 2014 to 2024, there is a consistent increase in related studies, with 66 selected articles, thus highlighting the rising interest in this topic and the importance that researchers and software engineering professionals have placed on this field.

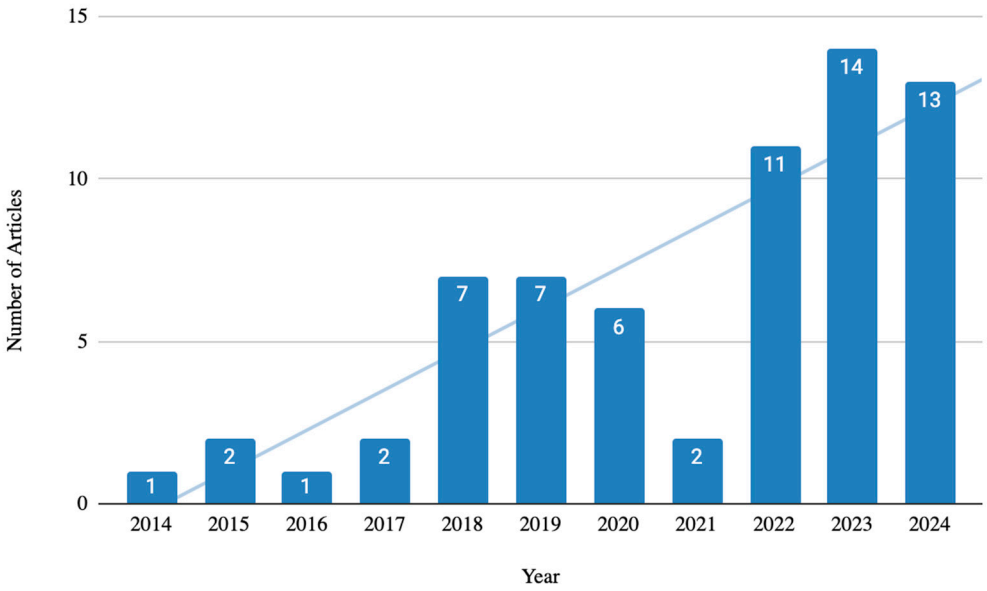


Figure 3. Numbers of publications over time.

Although the temporal evolution in Figure 3 was analyzed descriptively through frequency counts and visual trends, the purpose of this analysis was to illustrate the progressive growth of AI-related research in software testing rather than to perform inferential validation. The counts were normalized per year to ensure comparability, and the trend line reflects a consistent increase across the decade. Formal trend tests (e.g., Mann–Kendall or Spearman rank correlation) were not applied, since the aim of this review was exploratory and descriptive. Nevertheless, future studies could complement this analysis with statistical trend testing and confidence intervals to quantify uncertainty in the reported proportions and reinforce the robustness of temporal interpretations.

Figure 4 shows the journals in which the selected articles were published, classified by quartile and accompanied by the corresponding number of publications. In total, 28 articles were published in 10 journals with two or more publications. The journals contributing the most to the topic were IEEE Access and Information and Software Technology, both of which are ranked in Q1, with six and four articles, respectively. The category Others includes 38 articles distributed across 17 journals in Q1, seven in Q2, nine in Q3, and five in Q4, each contributing a single article. In total, 48 journals were examined, of which 36

were classified as Q1, reflecting the high quality and relevance of the sources considered in this study.

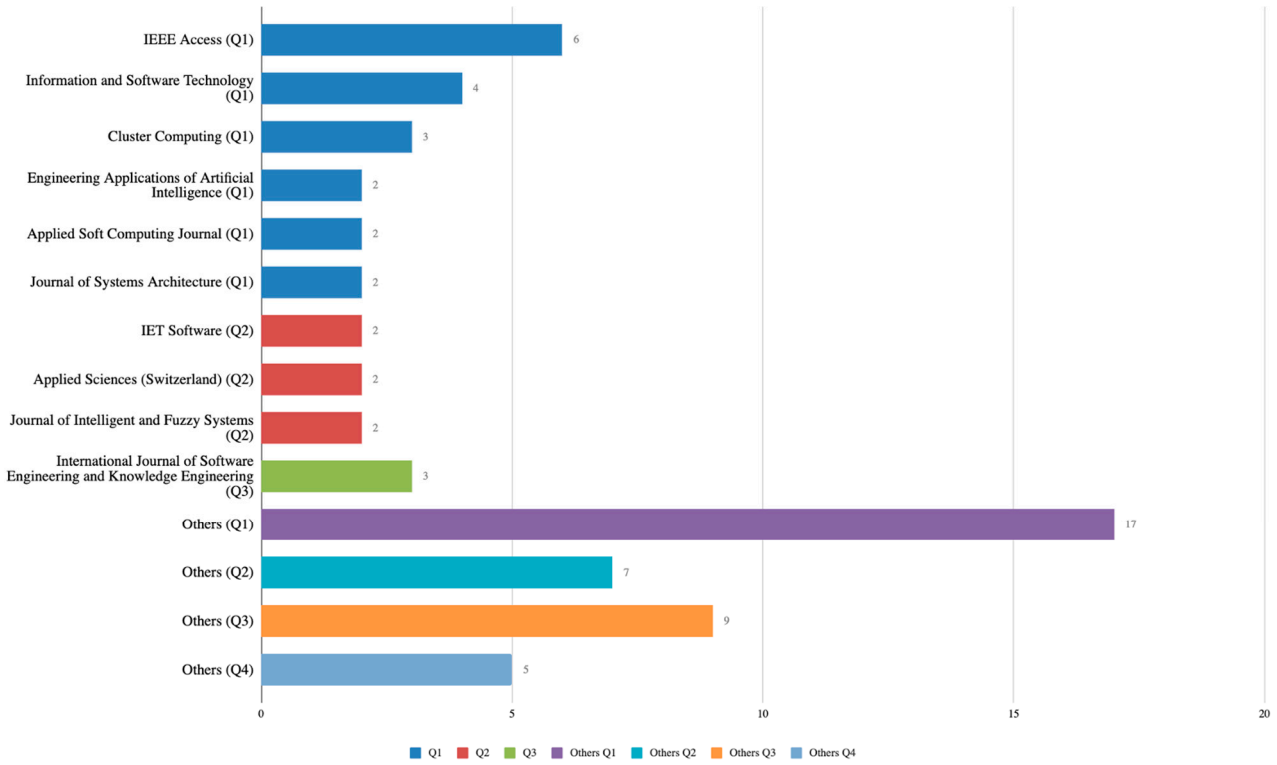


Figure 4. Journals reviewed by quartile.

Figure 5 illustrates the number of selected studies by quartile for this analysis. Notably, 54.5% of these correspond to articles published in Q1-ranked journals, illustrating the high quality of the data. This distribution highlights the robustness and relevance of the findings obtained in this research.

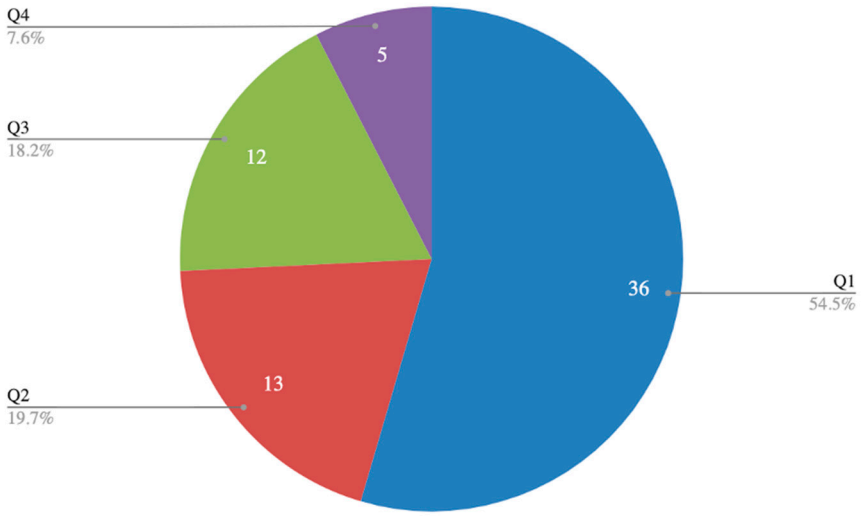


Figure 5. Articles selected by quartile.

The predominance of Q1 and Q2 journals among the selected studies indirectly reflects the high methodological rigor, peer-review standards, and overall credibility of the evidence base considered in this systematic review.

3.5. Analysis

3.5.1. RQ1: Which AI Algorithms Have Been Used in ST, and for What Purposes?

To ensure methodological consistency and avoid double-counting, the identification and classification of algorithms followed a structured coding process. Each algorithm mentioned across the selected studies was first normalized by its canonical name (e.g., "Random Forest" = RF, "Support Vector Machine" = SVM), and algorithmic variants (e.g., "Improved RF," "Hybrid RF–SVM") were mapped to their base algorithm family unless they introduced a new methodological contribution described by the authors as proposed.

Duplicates were resolved by cross-checking algorithm names within and across studies using the consolidated list in coding_book_taxonomy.xlsx. When the same algorithm appeared in multiple problem contexts (e.g., SDP and ATE), it was counted once for its family but associated with multiple application categories. Of the 66 selected studies a total of 332 unique algorithmic implementations were thus identified, of which 96 were novel proposals and 236 were previously existing algorithms reused for comparison. This classification ensures reproducibility and consistency across the dataset and Supplementary Materials.

To better understand these algo-rhythms, classification is necessary. It is worth noting that 14 algorithms appeared in both the novel and existing categories.

However, no study was found that proposed a specific taxonomy for these, and a classification based on forms of ST is not applicable, since some categories overlap. For example, a fully automated ST process (classified by the degree of automation) may also be functional (classified by test type). This indicates that the conventional forms of ST are not a suitable criterion for classifying AI algorithms and highlights the need for a new taxonomy.

After reviewing the identified algorithms, we observed that each was designed to solve specific problems within ST. This suggested that a classification based on the testing problems addressed by these algorithms would be more appropriate. In view of this, Table 4 presents the main problems identified in ST, which may serve as the foundation for a new taxonomy of AI algorithms applied to ST. This classification provides a precise and useful framework for analyzing and applying these algorithms in specific testing contexts, enabling optimization of their selection and use according to the needs of the system under evaluation.

Table 4. Taxonomy of AI Algorithms based on Software Testing.

ID	Category	Problem	Description	Source
SDP	Prediction	Software defect prediction	Predicts the likelihood of defects in the software before deployment	[121,122]
		Test report severity prediction	Estimates the severity of detected defects to prioritize their resolution	[123]
		Unstable test case prediction	Detects test cases likely to fail due to environmental changes	[124]
SDD	Detection	Software defect detection	Identifies specific defects in the source code during development	[125]
TCM	Test case management	Test case classification	Groups and categorizes test cases based on criteria such as complexity and risk	[87]
		Test case prioritization	Ranks test cases based on the importance and likelihood of detecting critical faults	[126]
		Automatic test case generation	Automatically generates test cases based on requirements and usage conditions	[127]

Table 4. *Cont.*

ID	Category	Problem	Description	Source
		Test case optimization	Improves test case efficiency by removing redundancies and maximizing coverage	[128]
ATE	Automation and execution	Software test data generation	Creates test data to simulate various usage conditions and verify software robustness	[88]
		UI test automation	Focuses on automating user interfaces for regression and functional testing	[94]
		Test code generation	Automatically generates the code required to execute specific tests	[112]
		Automated test Execution	Enables automatic execution of tests without manual intervention	[129]
CST	Collaboration	Collaborative GUI software testing	Supports collaborative GUI testing through shared tools	[81]
STC	Test coverage	GUI test coverage	Assesses the effectiveness of tests on graphical user interfaces	[82]
		Software test coverage	Measures how well the tests cover the software source code	[130]
STE	Test evaluation	Mutation tests	Uses mutations in source code to evaluate the ability of tests to detect changes	[80]
		Software security Testing	Focuses on identifying and mitigating security vulnerabilities in software	[131]
OTH	Other algorithms	Software fault mutant prediction	Estimates the likelihood of detecting specific defect mutations in software	[66]
		Software and video game test process automation	Automates end-to-end testing workflows for both software and video game environments	[86,104]
		Combinatorial software testing	Applies combinatorial techniques to design minimal yet comprehensive test case sets	[109]
		Software integration test ordering	Defines the optimal sequence for executing integration tests to improve fault detection efficiency	[111]

To strengthen the transparency and reproducibility of the proposed taxonomy, each category (e.g., TCM, ATE, STR, DEM, VI) was defined through explicit operational criteria derived from the problem–variable–metric relationships identified during the data extraction stage. Ambiguities or overlaps between categories were resolved by consensus between the two reviewers, following a structured coding guide that prioritized the dominant research objective of each study. The "Other" category included a limited number of interdisciplinary studies that did not fully fit within the main taxonomy dimensions but were retained to preserve representativeness. Although a formal inter-rater reliability coefficient (e.g., Cohen's κ) was not computed, complete agreement was achieved after iterative verification and validation in Microsoft Excel, ensuring traceability and methodological rigor throughout the classification process.

3.5.2. AI Algorithms in Software Defect Prediction

In this category, a total of 229 AI algorithms were identified as being applied to software defect prediction (SDP). Of these, 40 distinct algorithms were proposed in the papers, while 146 distinct algorithms were not novel. In addition, 25 novel hybrid algo-

rithms and 18 existing hybrid algorithms were identified, with 11 algorithms appearing in both categories.

Hybrid algorithms combine two or more individual algorithms and are identified using the "+" symbol. For example, C4.5 + ADB represents a combination of the individual algorithms C4.5 and ADB. Singular algorithms are represented independently, such as SVM, or with variants indicated using hyphens, such as KMeans-QT. In some cases, they may include combinations enclosed in parentheses, such as 2M-GWO (SVM, RF, GB, AB, KNN), indicating an ensemble or multi-model approach.

Table 5 summarizes the AI algorithms proposed or applied in each study, as well as the existing algorithms used for comparative evaluation.

Table 5. Algorithms in SDP.

Study	Novel Algorithm(s)	Existing Algorithm(s)
[20]	RNNBDL	LSTM, BiLSTM, CNN, SVM, NB, KNN, KStar, Random Tree
[59]	2M-GWO (SVM, RF, GB, AB, KNN)	HHO, SSO, WO, JO, SCO
[60]	ANN, SVM	n/a
[61]	LineFlowDP (Doc2Vec + R-GCN + GNNExplainer)	CNN, DBN, BoW, Bi-LSTM, CodeT5, DeepBugs, IVDetect, LineVD, DeepLineDP, N-gram
[64]	HFEDL (CNN, BiLSTM + Attention)	n/a
[65]	IECGA (RF + SVM + NB + GA)	RF, SVM, NB
[67]	VESDP (RF + SVM + NB + ANN)	RF, SVM, NB, ANN
[68]	PoPL(Hybrid)	n/a
[69]	bGWO (ANN, DT, KNN, NB, SVM)	ACO
[70]	FMR, FMRT	NB, RF, ACN, ACF
[71]	CNN	n/a
[73]	LM, BP, BR, BR + NN	SVM, DT, KNN, NN
[74]	DEPT-C, DEPT-M1, DEPT-M2, DEPT-D1, DEPT-D2	DE, GS, RS
[75]	MLP, BN, Lazy IBK, Rule ZeroR, J48, LR, RF, DStump, SVM	n/a
[76]	C4.5 + ADB	ERUS, NB, NB + Log, RF, DNC, SMT + NB, RUS + NB, SMTBoost, RUSBoost
[77]	CONVSDP (CNN), DNNSDP (DNN)	RF, DT, NB, SVM
[78]	ISDPS (NB + SVM + DT)	NB, SVM, DT, Bagging, Vouting, Stacking
[79]	DT, NB, RF, LSVM	n/a
[83]	KPCA + ELM	SVM, NB, LR, MLP, PCA + ELM
[84]	WPA-PSO + DNN, WPA-PSO + self-encoding	Grid, Random, PSO, WPA
[85]	ACO	NB, J48, RF
[90]	MODL-SBP (CNN-BiLSTM + CQGOA)	SVM-RBF, KNN + EM, NB, DT, LDA, AdaBoost,
[91]	KELM + WSO	SNB, FLDA, GA + DT, CGenProg
[92]	DP + GCNN	LRC, RFC, DBN, CNN, SEML, MPT, DP-T, CSEM
[93]	MLP	n/a
[95]	Flakify (CodeBERT)	FlakeFlagger
[96]	MVFS (MVFS + NB, MVFS + J48, MVFS + IBK)	IG, CO, RF, SY

Table 5. *Cont.*

Study	Novel Algorithm(s)	Existing Algorithm(s)
[97]	rejoELM, IrejoELM	rejoNB, rejoRBF
[99]	CCFT + CNN	RF, DBN, CNN, RNN, CBIL, SMO
[100]	Naïve Bayes (GaussianNB)	n/a
[101]	Stacking + MLP (J48, RF, SMO, IBK, BN) + BF, GS, GA, PSO, RS, LFS	n/a
[103]	TS-ELA (ELA + IG + SMOTE + INFFC) + (BaG, RaF, AdB, LtB, MtB, RaB, StK, StC, VoT, DaG, DeC, GrD, RoF)	DTa, DSt
[105]	CBA2	C4.5, CART, ADT, RIPPER, DT
[107]	HyGRAR (MLP, RBFN, GRANUM)	SOM, KMeans-QT, XMeans, EM, GP, MLR, BLR, LR, ANN, SVM, CCN, GMDH, GEP, SCART, FDT-O, FDT-E, DT-Weka, BayesNet, MLP, RBFN, ADTree, DTbl, CODEP-Log, CODEP-Bayes
[108]	KTC (IDR + NB, IDR + SVM, IDR + KNN, IDR + J48)	NB, KNN, SVM, J48
[115]	SDP-CMPOA (CMPOA + Bi-LSTM + Deep Maxout)	CNN, DBN, RNN, SVM, RF, GH + LSTM, FA, POA, PRO, AOA, COOT, BES
[117]	2SSEBA (2SSSA, ELM, Bagging Ensemble)	ELM, SSA + ELM, 2SSSA + ELM, KPWE, SEBA
[119]	ME-SFP + [DT], ME-SFP + [MLP]	Bagging + DT, Bagging + MLP, Boosting + DT, Boosting + MLP, Stacking + DT, Stacking + MLP, Indi + DT, Indi + MLP, Classic + ME
[120]	AST n-gram + J48, AST n-gram + Logistic, AST n-gram + Naive Bayes	n/a

3.5.3. AI Algorithms in SDD, TCM, ATE, CST, STC, STE and Others

In these categories, a total of 103 AI algorithms were identified, which were distributed as follows:

- In the SDD category, eight algorithms were found, of which two were novel (one singular and one hybrid), six were existing (all singular), and one was repeated.
- In the TCM category, 28 algorithms were identified, including 10 novel singular algorithms, 18 existing (15 singular and three hybrid), and one repeated.
- The ATE category comprised 21 algorithms, of which six were novel (four singular and two hybrid), 14 existing (all singular), and one repeated.
- In the CST category, four algorithms were identified: one novel and three existing, with no hybrids or repetitions. The STC category included 18 algorithms: four novel (three singular and one hybrid), 14 existing (all singular), and no repetitions.
- For the STE category, seven algorithms were found: three novel (two singular and one hybrid), one existing (singular), and no repetitions.
- In the OTH category, 17 algorithms were identified: five novel (all singular), and 12 existing (all singular), with no repetitions.

Table 6 provides a consolidated summary of the novel and existing algorithms identified in each category.

Table 6. Algorithms in SDD, TCM, ATE, CST, STC, STE, and OTH.

Category	Study	Novel Algorithm(s)	Existing Algorithm(s)
SDD	[18]	SVM + MLP + RF	SVM, ANN, RF

Table 6. *Cont.*

Category	Study	Novel Algorithm(s)	Existing Algorithm(s)
	[106]	FRBS	C4.5, RF, NB
TCM	[17]	EfficientDet, DETR, T5, GPT-2	n/a
	[19]	T5 (YOLOv5)	n/a
	[62]	XCSF-ER	ANN, RS, XCSF
	[72]	MFO	FA, ACO
	[98]	IFROWANN av-w_1	EUSBoost, SMOTE + C4.5, CS + SVM, CS + C4.5
	[110]	KNN	LR, LDA, CART, NB, SVM
	[118]	AFSA	GA, K-means clustering, NSGA-II, IA
ATE	[63]	SFLA	GA, PSO, ACO, ABC, SA
	[87]	NN (LSTM + MLP)	Hierarchical Clustering
	[88]	ACO + NSA	Random testing, ACO, NSA
	[94]	EfficientNet-B1	CNN, VGG-16, ResNet-50, MobileNet-V3
	[112]	NMT	n/a
	[116]	RL-based-CI	RL-BS1,RL-BS2
CST	[81]	ERINet	SIFT, SURF, ORB
STC	[82]	HashC-NC	NC, 2-way, 3-way, INC, SC, KMNC, HashC-KMNC, TKNC
	[113]	ER-Fuzz (Word2Vec + LSTM)	AFL, AFLFast, DT, LSTM
	[114]	NSGA-II, MOPSO	Single-objective GA, PSO
STE	[80]	MTUL (Autoencoder)	n/a
	[89]	CVDF DYNAMIC (Bi-LSTM + GA)	NeuFuzz, VDiscover, AFLFast
	[102]	ARTDL	RT
OTH	[66]	FrMi	SVM, RF, DT, LR, NB, CNN
	[86]	MLP	Random strategy, total strategy, additional strategy
	[104]	LSTM	n/a
	[109]	MiTS	n/a
	[111]	RL	GA, ACO, RS

3.5.4. RQ2: Which Input Variables Are Used by AI Algorithms in ST?

In the context of this systematic review, the term variable refers exclusively to the input data that are used to feed AI algorithms in ST tasks. These variables originate from the datasets used in the studies reviewed here and represent the observable features that define the problem to be solved. They should not be confused with the internal parameters of the algorithms (such as learning rate, number of neurons, or trees), nor with the evaluation metrics used to assess the model performance (e.g., precision, recall, or F1-score), which are addressed in RQ3.

These input variables are important, as they determine how the problem is represented, and hence directly influence the model training process (see Figure 6), its generalization capability, and the quality of the predictions. For instance, in the case of software defect prediction, it is common to use metrics extracted from the source code, such as the cyclomatic complexity or the number of public methods.



Figure 6. Data algorithm and models used in software testing.

Based on an analysis of the selected studies, a total of 181 unique variables were identified, which were organized into a taxonomy of ten thematic categories. This classification provided a clearer understanding of the different types of variables used, their nature, and their source. Table 7 presents a consolidated summary: for each category, it shows the identified subcategories, the total number of variables, the number of associated studies, and the corresponding reference codes. A detailed list of these variables can be found in Appendix C.

Table 7. AI input Variables used in ST.

Category	Subcategory	# Variable	# Studies	Studies
SCM: Structural source code metrics	Structural code metrics, OO metrics, syntactic metrics, integration/OO dependency structure, static code metrics	64	41	[18,20,59,60,64,65,67–71,73–78,83–85,88,90–97,99–101,105–107,111,115,117–120]
CQM: Complexity/ quality metrics	Halstead metrics, Halstead-like metrics (or alternatives), software quality metrics, concurrency metric	37	28	[18,20,59,65,67,69,70,73,74,76–79,83–85,91,96,97,99–101,103,105–107,119,120]
EHM: Evolutionary/ historical metrics	Change history, defect history, change metrics, multi-source (history + code), programs, test sets, combinatorial structure	25	11	[20,59,68,73,74,86,96,109,114,115,118]
DEM: Dynamic/ execution metrics	Execution dynamics, traces and calls, mutant execution metrics, MPI communication	22	6	[60,62,66,79,87,112]
STR: Semantic/textual representation	Textual semantics, embedded representation, BDD scenario/text, descriptive Statistics	20	9	[60,61,79,82,89,98,113,115,116]
VIM: Visual/interface metrics	Visuals/images, GUI visuals/interface processing, GUI interaction, interface elements, graphical models/ state diagrams	6	8	[17,19,72,81,94,102,110,114]
SBT: Search-based testing/fuzzing	Search-based fuzzing, fuzzing	2	1	[89]
STM: Sequential and temporal models	Temporal sequence (interaction), latent representations (auto-encoding)	2	2	[80,104]
NCM: Network/ connectivity metrics	Network metrics	2	1	[61]
SLC: Supervised labeling and classification	Supervised labeling	1	1	[113]

3.5.5. RQ3: Which Metrics Are Used to Evaluate the Performance of AI Algorithms in ST?

Table 8 summarizes the metrics employed in the primary studies to evaluate the performance of AI algorithms when applied to ST. These metrics have been organized into six evaluation disciplines to enable a better understanding not only of their frequency of use but also of their functional purpose across different evaluation contexts. A total of 62 distinct metrics were identified. A detailed list, including definitions and the studies that used them, is available in Appendix D.

Table 8. AI Algorithm Metrics for evaluating ST.

Category	Description	Metrics	# Metric	# Studies	Studies
CP: Classical performance	Evaluate classification accuracy and sensitivity	Accuracy, precision, recall, F1-score	4	38	[18,20,60,64–69,71,73–79,83,84,87,89–94,97,99–101,103,105,107,110,113,115,119,120]
AC: Advanced classification	Robust measures for class imbalance and comparative analysis	MCC, ROC-AUC, balanced accuracy, G-mean,	4	26	[20,59,61,64–66,74,76,77,83–85,90–93,96,98,101,103,105,107,114,117,119,120]
CE: Cost/error and probabilistic metrics	Quantify continuous prediction errors or losses	Brier Score, D2H, RMSE, ETT_instance, ETT_recall, Misclassification Rate	6	6	[67,74,78,103,106,107]
AR: Alarms and risk	Assess false positives, sensitivity, and specificity	Specificity (TNR), NPV, FDR, FNR, FPR, TPR, TNR, PD, PF	9	14	[67,70,73,76,78,87,89,91,100,105,107,115,117,119]
STS: Software testing specific metrics	Domain-specific: effort, localized coverage, and test case prioritization	Effort@Top20%recall, Recall@Top20%LOC, IFA, Top-k accuracy, KE, NAPFD, F-measure, MCA, Coverage_t-way, improvement	10	6	[20,61,86,102,109,110]
CGD: Coverage, execution, GUI, and deep learning	Measure structural coverage, neural activations, and GUI testing	APTC, EET, BLEU, mAP, total time (ms), redundancy (%), fraction of implemented steps, fraction of unimplemented steps, fraction of POM methods, AC, AG, SR, AT, correct rate, HashC coverage, NC, 2-way coverage, 3-way coverage, accuracy (coverage), coverage, code coverage, WDC, mutation score, L2, total stubs, saving rate, APFD, correct rate, balance score	29	16	[17,19,63,72,80–82,86,88,89,104,111,112,114–116,118]

For transparency and reusability, the proposed taxonomies of algorithms, input variables, and evaluation metrics are formally defined and documented. The detailed operational definitions, coding rules, and representative examples for each category are provided in the Supplementary Material on the file: coding_book_taxonomy.xlsx.

4. Evolution of AI Algorithms in ST

This section examines the evolution of AI algorithms applied to ST. The process used to explore this evolution was structured into three key stages, reflecting the methodology employed, the development of the investigation, and the main results. Each of these stages is described in detail below.

4.1. Method

To analyze the evolution of AI algorithms in ST, the following methodological phases were implemented:

- Phase 1—Algorithm Inventory

The AI algorithms that have been applied to ST are collected and cataloged based on the specialized literature.

- Phase 2—Aspects

The aspects to be analyzed are identified to explore the evolution of the algorithms listed in Phase 1.

- Phase 3—Chronological Behavior

The AI algorithms are organized chronologically, according to the aspects defined in Phase 2.

- Phase 4—Evolution Analysis

The changes and trends in the use of AI algorithms in ST are examined over time, based on each identified aspect.

- Phase 5—Discussion

The findings are discussed with their implications in terms of the observed evolutionary patterns.

4.2. Development

Phase 1. As detailed in Section 3, an exhaustive review of the specialized literature on AI algorithms in ST was conducted, in which we identified 332 algorithms across 66 selected studies. These were classified into 21 problems, which were further organized into eight categories: software defect prediction (SDP), software defect detection (SDD), test case management (TCM), test automation and execution (ATE), collaboration (CST), test coverage (STC), test evaluation (STE), and others (OTH) (see Table 4).

Phase 2. Three key aspects were identified for analysis:

- ST Problems: This refers to the categories of algorithms oriented toward specific testing problems.
- ST Variables: This represents the input variables related to the datasets used in the studies.
- ST Metrics: These are the evaluation metrics used by the algorithms to assess their performance.

An inventory was compiled from the summary data presented in Tables 5–8. This inventory identified:

- 66 studies in which AI algorithms were applied to ST problems.
- 108 instances involving the use of input variables across the 66 selected studies. Since a single study may contribute to multiple categories, the total number of instances exceeds the number of unique studies.

- 106 instances in which evaluation metrics were employed across the same set of studies. Again, the difference reflects overlaps where one study reported results in more than one metric category.

Table 9 provides a consolidated overview of the relationships among AI algorithms, problem categories, input variables, and evaluation metrics in software testing. Unlike previous figures that illustrated these dimensions separately, this table integrates them into a unified framework, allowing the identification of consistent research patterns and cross-dimensional connections. Each entry lists the corresponding literature codes [n], which facilitates traceability to the original studies while avoiding redundancy in naming all algorithms explicitly. This representation not only highlights the predominant associations—such as defect prediction with structural and complexity metrics evaluated through classical performance measures—but also captures emerging and exploratory combinations across less frequent categories. By mapping algorithms to problems, variables, and metrics simultaneously, Table 9 serves as the foundation for the integrative analysis presented in Section 5.4. The acronyms used in this figure correspond to the categories described in Tables 4, 7 and 8.

Table 9. Relationships between Problems, Variables and Metrics.

Algorithm	Problem	Variable	Metric
[20,59–61,64,65,67–71,73–79,83–85,90–93,95–97,99–101,103,105,107,108,115,117,119,120]	SDP	SCM, CQM, EHM, DEM, STR, NCM	CP, AC, CE, AR, STS, CGD
[18,106]	SDD	SCM, CQM	CP
[17,19,62,72,98,110,118]	TCM	SCM, EHM, DEM, STR, VIM	CP, AC, STS, CGD
[63,87,88,94,112,116]	ATE	SCM, DEM, STR, VIM	CP, AR, CGD, CGD
[81]	CST	SCM, VIM	CGD
[82,113,114]	STC	EHM, STR, VIM, SLC	CP, AC, CGD
[80,89,102]	STE	STR, VIM, SBT, STM	CP, AR, STS, CGD
[66,86,104,109,111]	OTH	SCM, EHM, DEM, STM	CP, AC, STS, CGD

A description of the algorithms used in each and information on the variables and evaluation metrics is provided in Appendices B–D. In addition, the dataset, the evaluated instances, and the performance results for each algorithm can be found in the path: https://github.com/escalasoft/ai-software-testing-review-data (accessed on 3 November 2025).

Phase 3. The algorithms were classified according to the three aspects under analysis, and their changes and trends over time were examined. The results are presented in Section 4.3.

Phase 4. The results obtained in Phase 3 were analyzed and interpreted, and a discussion is provided in Section 5.

4.3. Evolution of IA Algorithms and Their Application Categories in Software Testing

Figure 7 illustrates how the different problem categories in ST evolved from 2014 to 2024. The vertical axis shows the seven identified problem categories in ST, along with an additional category labeled Other (OTH) to represent miscellaneous problems. The horizontal axis displays the year of publication.

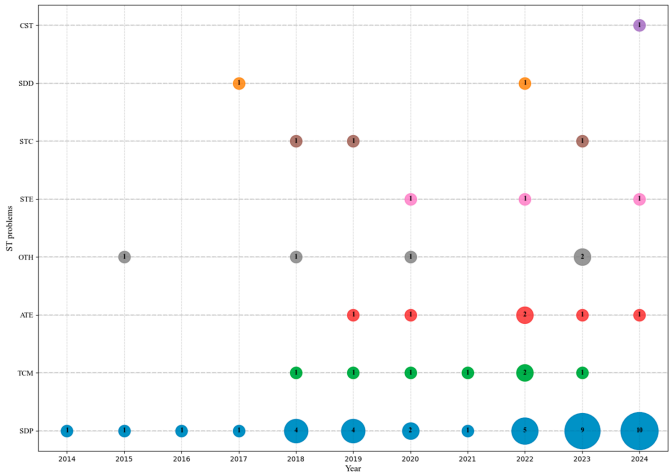


Figure 7. Evolution of AI algorithms in software testing problem domains. Each bubble represents the number of studies associated with a specific algorithm category, where the bubble size is proportional to the total count of studies. The color of each bubble denotes the problem domain: blue = Software Defect Prediction (SDP), green = Test Case Management (TCM), red = Automation and Execution of Testing (ATE), lead = Otros (OTH), pink = Software Test Evaluation (STE), brown = Software Test Coverage (STC), orange = Software Defect Detection (SDD), and purple = Collaboration Software Testing (CST).

These studies reveal a clear research trend in the application of AI algorithms to various ST problems. For instance, the software defect prediction (SDP) category stands out as the most extensively addressed, while the automation and execution of testing (ATE) and test case management (TCM) categories show a promising upward trend in recent year.

This visualization highlights the relative research intensity and prevalence of each algorithm within the software testing domain.

The vertical axis of Figure 8 shows the distribution of 10 categories of software testing input variables, which are grouped based on their structural, semantic, dynamic, and functional characteristics. These categories reveal a significant evolution over the past decade. The horizontal axis represents the year of publication of the studies.

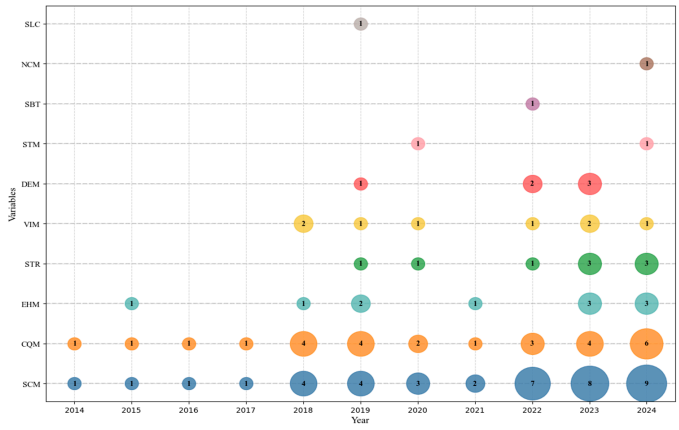


Figure 8. Evolution of IA algorithms in relation to software testing variables. Each bubble represents the number of studies within a given variable category, where bubble size corresponds to the total number of studies, and color indicates the related metric domain: blue = Structural Code Metrics (SCM), orange = Complexity Quality Metrics (CQM), sky blue = Evolutionary Historical Metrics (EHM), green = Semantic Textual Representation (STR), yellow = Visual Interface Metrics (VIM), red = Dynamic Execution Metrics (DEM), pink = Sequential Temporal Models (STM), purple = Search Based Testing (SBT), brown = Network Connectivity Metrics (NCM), and lead Supervised Labeling Classification (SLC).

To illustrate the evolution in the usage of evaluation metrics, the vertical axis of Figure 9 displays the six metric disciplines applied in AI-based ST, while the horizontal axis represents the year of publication. It can clearly be seen that most studies employ classical performance metrics (CPs), such as accuracy, precision, recall, and F1-score, as well as those within the advanced classification discipline (AC), which includes indicators such as MCC, ROC-AUC, balanced accuracy, and G-mean.

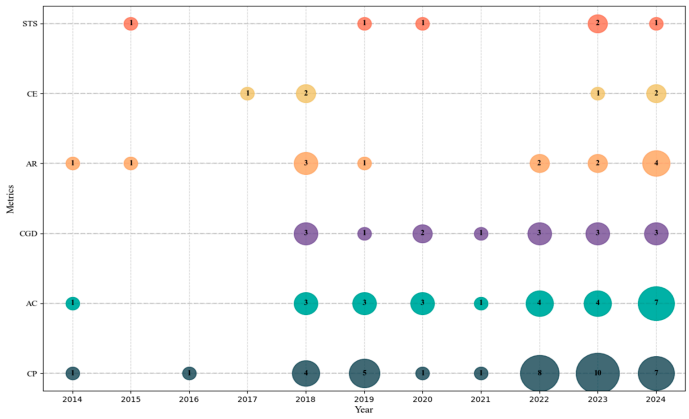


Figure 9. Evolution of AI algorithms with respect to software testing metrics. Each bubble represents the number of studies using a particular evaluation metric, where bubble size reflects the total count of studies and color differentiates the metric groups: lead = Classical Performance (CP), green = Advanced Classification (AC), purple = Coverage GUI Deep Learning (CGD), orange = Alarms and Risk (AR), yellow = Cost Error (CE), and dark orange = Software Testing Specific (STS).

Limitations and Validity Considerations

Although the evolution of AI algorithms in software testing has been systematically analyzed, this study is not exempt from potential limitations. Regarding construct validity, the taxonomy and classification trends were derived from existing studies and may not fully represent emerging paradigms. Concerning internal validity, independent screening and consensus-based extraction aimed to reduce bias, though subjective interpretation during categorization may have influenced some patterns.

In terms of external validity, the analysis was restricted to peer-reviewed journal publications indexed in Scopus and Web of Science, which may exclude newer conference papers that could reflect recent industrial practices. Finally, conclusion validity may be affected by dataset heterogeneity and publication bias. These issues were mitigated through rigorous inclusion criteria, adherence to PRISMA 2020 recommendations, and transparent reporting to ensure reproducibility and reliability of the synthesis.

5. Discussion

AI algorithms play a crucial role in ST, a key component of the software development lifecycle that directly affects the quality of the final product. In view of their importance, it is essential to analyze and discuss how these algorithms have evolved and their contributions to ST over time.

5.1. Evolution of Algorithms in Software Testing Problems

Our analysis of the evolution of AI algorithms applied to software testing (ST) problems reveals a growing emphasis on automation, optimization, and process enhancement across different stages of the ST lifecycle. From our classification of these problems into eight main categories, a progressive maturation of research approaches in this field is evident.

First, software defect prediction (SDP) has historically been the most dominant category. This research stream has focused on estimating the likelihood of defects occurring prior to deployment, as well as predicting the severity of test reports to enable more effective prioritization. Its persistent use over time underscores the continued relevance of this approach in contexts where software quality and reliability are critical.

Software defect detection (SDD) has recently gained more attention, targeting not only the prediction of unstable failures but also the direct identification of defects at the source code level. This reflects the growing need for intelligent systems capable of detecting issues before they reach production, thereby strengthening quality assurance.

A particularly noteworthy trend is the expansion of the test case management (TCM) category, which includes problems related to the prioritization, generation, classification, execution, and optimization of test cases. Its sustained growth in recent years reflects increasing interest in leveraging AI solutions to scale, automate, and streamline validation activities, particularly within agile and continuous integration environments.

Progress has also been observed in the automation and execution of tests (ATE) category, which ranges from UI automation to the automatic generation of test data and code. This category has become more prominent with the rise of generation techniques such as code synthesis and test data creation, which reduce manual effort and accelerate testing cycles.

The collaborative software testing (CST) category, which focuses on the collective and coordinated management of testing activities, has emerged as an incipient yet promising area. Supported by collaborative platforms and shared tools, this approach suggests an evolution toward more distributed and cooperative testing practices.

Test coverage (STC) remains a less frequent but relevant dimension, especially in evaluating the effectiveness of tests over source code or graphical interfaces. Its integration with AI has enabled the identification of uncovered areas and improvements in the design of automated test strategies.

Finally, the test evaluation (STE) category, which encompasses mutation testing and security analysis, has also advanced significantly in the past five years. These methodologies facilitate the assessment of the robustness of generated test suites and their ability to detect changes or vulnerabilities in the system.

Other problems (OTH) group heterogeneous tasks that do not neatly fit the previous families but are relevant to the evolution of AI in ST. Examples include integration test ordering, mutation-specific defect prediction, automated end-to-end testing workflows (e.g., for game environments), software process automation, and combinatorial test design. Although less frequent, this category captures emerging or domain-specific applications and preserves completeness without forcing weak assignments to other families.

In summary, the evolution of ST problem categories shows a transition from classical defect-centric approaches (SDP, SDD) toward more sophisticated strategies that span the entire testing value chain (TCM, ATE, CST), while also incorporating collaborative (STC) and evaluation-oriented (STE) dimensions, together with a residual OTH group that reflects emergent and domain-specific tasks. This diversification indicates that the application of AI in ST has not only intensified but also matured to embrace multidisciplinary approaches and adapt to increasingly complex operational contexts.

5.2. Evolution of Algorithms Regarding Software Testing Variables

The analysis of input variables used to train AI algorithms in software testing reveals a progressive diversification over the last decade. A total of 10 categories of variables were identified, each contributing distinct perspectives on how testing problems are represented and addressed. These categories are: Structural Code Metrics (SCM), Complex-

ity/Quality Metrics (CQM), Evolutionary/Historical Metrics (EHM), Dynamic/Execution Metrics (DEM), Semantic/Textual Representation (STR), Visual/Interface Metrics (VIM), Search-Based Testing/Fuzzing (SBT), Sequential and Temporal Models (STM), Network/Connectivity Metrics (NCM), and Supervised Labeling/Classification (SLC).

A closer look at their evolution allows us to distinguish three stages:

2014–2017: Foundation on SCM and CQM.

Research in this initial stage was largely dominated by structural code metrics (e.g., size, complexity, cohesion, coupling) and complexity/quality metrics (e.g., Halstead or McCabe indicators). These variables were critical for early AI-based models, providing a static view of the software structure and code quality.

2018–2020: Expansion toward EHM, DEM, and STR.

As testing scenarios became more dynamic, the field incorporated evolutionary/historical metrics (e.g., change history, defect history), dynamic/execution metrics (e.g., traces, execution time, call frequency), and semantic/textual representations (e.g., bug reports, documentation, natural language descriptions). This transition reflects an interest in contextual and behavioral features that move beyond static code.

2021–2024: Diversification into emerging categories.

In the most recent stage, less explored but innovative categories gained relevance: visual/interface metrics (e.g., GUI features, graphical models), search-based testing and fuzzing, sequential and temporal models (e.g., recurrent patterns, autoencoders), network/connectivity metrics, and supervised labeling/classification. Although these categories appear with lower frequency, their emergence highlights novel approaches aligned with the complexity of modern software ecosystems, including mobile, distributed, and intelligent systems.

In summary, the evolution of input variables illustrates a transition from traditional static code-centric approaches (SCM and CQM) toward a multidimensional perspective that integrates historical, dynamic, semantic, and even network-oriented features. This shift demonstrates how AI in software testing has matured, not only broadening the range of variables but also adapting to the complexity of contemporary testing environments.

More recently, the emergence of categories such as VIM, STM, and NCM—reported only in a small number of studies between 2019 and 2024 (see Figure 8 and Table 7)—illustrates the diversification of input variables in AI-based software testing. These categories point to novel perspectives, such as visual interactions, temporal modeling, and network connectivity, which had not been addressed in earlier work. Their introduction has driven initial experimentation with hybrid and explainable AI approaches documented in the reviewed literature, particularly in contexts where capturing sequential dependencies, user interfaces, or connectivity is essential. Consequently, these studies often require more advanced performance metrics to evaluate robustness and generalization. Taken together, the findings indicate that the evolution of variables, algorithms, and metrics has been interdependent, with progress in one dimension enabling advances in the others.

5.3. Evolution of Algorithms in Software Testing Metrics

The evolution of AI algorithms in software testing also reflects a progressive refinement of the metrics employed to evaluate their performance, robustness, and practical applicability. Based on the reviewed studies, six main categories of metrics were identified, ranging from classical evaluation to testing-specific measures (Table 8, Figure 9). Initially, research was dominated by classical performance (CP) metrics, such as accuracy, precision, recall, and F1-score. These measures, particularly linked to prediction tasks, provided the most accessible foundation for assessing algorithmic capacity and comparability, although they often fall short in capturing robustness or scalability in complex contexts.

From 2018 onward, studies began incorporating advanced classification (AC) metrics, including MCC, ROC-AUC, and balanced accuracy. These measures offered greater robustness in handling imbalanced datasets, a frequent issue when predicting software defects. Their adoption illustrates a methodological shift toward richer and more nuanced evaluation strategies, which became more prevalent as algorithms diversified in scope.

A further development was the introduction of cost/error (CE) metrics, alarms and risk (AR) indicators, and coverage/execution/GUI-driven (CGD) metrics. These reflected the community's growing interest in evaluating algorithms not only on accuracy but also on their operational impact, error sensitivity, and ability to capture the completeness of testing processes. Similarly, software testing-specific (STS) measures were adopted to directly benchmark AI methods against domain-grounded baselines, ensuring fairer assessments across heterogeneous testing scenarios.

Periodization of metric evolution reveals three distinct phases.

2014–2017: Early studies relied almost exclusively on classical performance (CP) metrics, focusing on accuracy and recall as the standard for validating predictive models.

2018–2020: The field expanded to advanced classification (AC) and cost/error (CE) metrics, reflecting the need to handle imbalanced datasets and quantify error propagation more precisely.

2021–2024: There is a clear transition toward coverage-oriented (CGD) and testing-specific (STS) measures, alongside alarms and risk (AR) metrics. This diversification indicates the community's growing emphasis on robustness, scalability, and the operational reliability of AI-based testing in industrial contexts.

In summary, the evolution of metrics reveals a clear transition from general-purpose evaluation (CP) toward more robust, domain-specific, and context-aware approaches (STS, CGD). This trend underscores the growing need to align evaluation strategies with the complexity of AI models and the operational realities of modern software testing.

5.4. Integrative Analysis

As shown in Table 9, the relationships between algorithms, problem categories, input variables, and evaluation metrics reveal a complex interplay that goes beyond examining these dimensions in isolation. This integrative view enables the identification of consolidated research patterns as well as emerging directions in AI-based software testing. Figure 10 visualizes these relationships through three complementary heatmaps that illustrate the co-occurrence frequencies between problems, variables, and metrics across the 66 studies analyzed.

The heatmap also reflects the strength and nature of interdependencies among testing problems and algorithmic approaches. High-frequency associations, such as "SDP–SCM–CP," indicate mature research intersections where predictive models are frequently integrated with configuration management and change propagation processes. In contrast, low-frequency patterns like "VIM–CGD" suggest emerging or underexplored connections, potentially representing novel directions for integrating visual interface metrics with code generation defects.

In first place, defect prediction (SDP) continues to dominate the landscape, consistently associated with structural code metrics (SCM) and complexity/quality metrics (CQM). These studies are primarily evaluated through classical performance indicators (CP), such as accuracy, recall, and F1-score, reinforcing their maturity and long-standing presence in the field. The concentration of high-intensity cells in the heatmap confirms this consistent alignment between SDP, SCM/CQM, and CP, reflecting the community's confidence in leveraging structural code features for predictive purposes and highlighting the foundational role of SDP in establishing AI as a viable tool for software testing. This is important

because it allows this combination of components to continue addressing SDP-related cases within the testing development cycle. At present, companies that recognize this frequency may incorporate similar features into their own testing models, thereby strengthening SDP-related practices and enhancing overall productivity across the software development and quality assurance cycle.

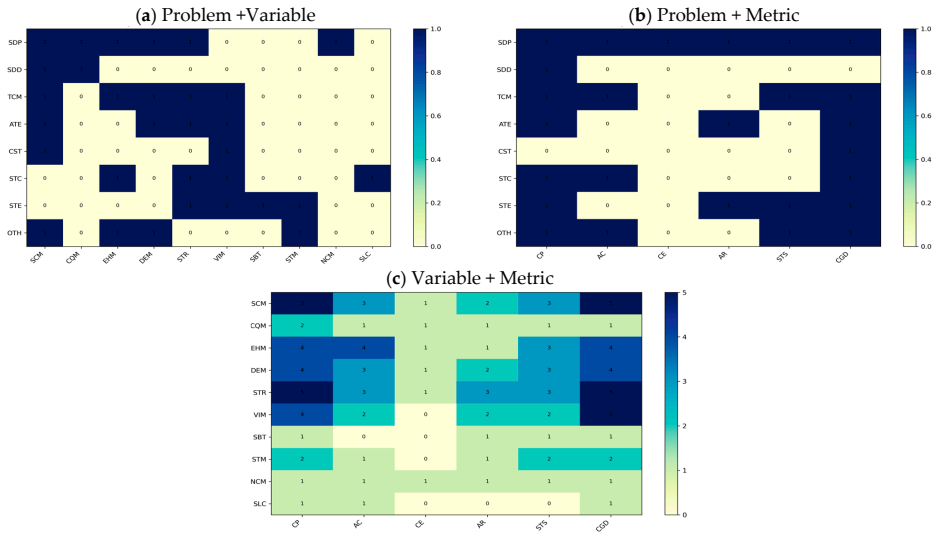


Figure 10. Integrative heatmaps of AI algorithms in software testing. The color intensity represents the frequency of co-occurrence across the 66 studies analyzed.

In second place, the categories of automation and execution of tests (ATE) and test case management (TCM) show significant expansion. ATE demonstrates a broader combination of input variables, particularly dynamic execution metrics (DEM) and semantic/textual representations (STR). Their evaluation increasingly relies on advanced classification (AC) and coverage-oriented (CGD) metrics, evidencing a shift toward more sophisticated and realistic testing environments. However, the datasets maintained by companies may contain noise depending on their specific business domain, which could lead to uncertainty regarding their implementation reliability. Moreover, hasty decisions derived from the low recurrence of these metrics could negatively impact short-term return on investment. Meanwhile, TCM is frequently linked with evolutionary/historical variables (EHM) and semantic/textual features (STR). Its evaluation integrates both classical and advanced metrics, underscoring its evolution toward scalable solutions for prioritization, optimization, and automation in agile and continuous integration contexts. These tendencies are clearly visible in the heatmaps, where clusters combining DEM/STR with AC/CGD emphasize the strong methodological coupling that supports the expansion of ATE and TCM research. This is particularly relevant for industrial development teams that are continuously exploring ways to maximize the efficiency of their models. An effective combination of metrics is essential for organizations to sustain key performance indicators and mitigate productivity risks throughout the software testing lifecycle.

In third place, emerging categories such as collaborative testing (CST), test evaluation (STE), and the integration of sequential/temporal models (STM), network connectivity metrics (NCM), and visual/interface metrics (VIM) appear less frequently but add methodological diversity. These approaches often combine heterogeneous variables and metrics, addressing challenges such as distributed systems, time-dependent fault detection, security validation, and usability assessment. Although less consolidated, they represent innovative directions that could expand the scope of AI-based software testing soon. This trend is also supported by the heatmaps, which display lighter but distinct links between VIM and CGD

as well as STM and STS, suggesting emerging but still underexplored lines of investigation. The significance of these tendencies provides valuable input for academia–industry collaborations, which could leverage these findings to design high-impact research initiatives and foster the creation of innovative products that contribute to the virtuous cycle of scientific and technological advancement.

Finally, categories grouped under "Other" (OTH) illustrate exploratory lines of research where algorithms are tested across varied and heterogeneous combinations of problems, variables, and metrics. While not yet mature, these contributions enrich the methodological landscape and open opportunities for cross-domain applications, particularly when combined with advances in explainability and hybrid AI approaches. The low but widespread co-occurrence patterns in the heatmaps visually confirm this experimental nature and highlight how these studies are paving new interdisciplinary bridges for future AI-driven testing frameworks.

Overall, this integrative analysis confirms that the evolution of AI algorithms in software testing cannot be fully understood without considering the interdependencies between the problems addressed, the nature of the input variables, and the evaluation strategies employed. Advances in one dimension—such as the refinement of variables or the design of new metrics—have consistently enabled progress in the others. This interdependence underscores the need for holistic frameworks that explicitly connect problems, variables, and metrics, thereby guiding the design, benchmarking, and industrial adoption of AI-based testing solutions.

However, the gap between academic research and industrial adoption remains one of the main challenges in applying AI-driven testing solutions. In industrial environments, models are often constrained by excessive noise in historical test data, incomplete labeling, and high operational costs associated with model deployment and maintenance. These factors limit the reproducibility of experimental results reported in academic studies. Furthermore, the lack of standard test environments and privacy restrictions on industrial data often prevent large-scale validation, making the transfer of research prototypes into production environments difficult.

Industrial Applicability and Maturity of AI Testing Approaches

While most of the reviewed studies emphasize academic contributions and their challenges in industry, several AI testing approaches have reached a level of maturity that enables industrial adoption. SDP and ATE techniques are highly deployable thanks to their integration with continuous integration pipelines, historical code metrics, and model-based testing tools. These approaches demonstrate reproducible performance and scalability across diverse projects, making them viable candidates for adoption in DevOps environments.

Conversely, categories such as Collaborative Software Testing (CST) and Test Evaluation (STE) still face significant practical barriers. Challenges arise from the lack of explainability (XAI) in complex AI models, limited interoperability with legacy testing infrastructures, and the absence of standard evaluation benchmarks for cross-organizational collaboration. Addressing these limitations requires closer collaboration between academia and industry, focusing on interpretability, scalability, and sustainable automation pipelines that can operate within real-world software ecosystems.

5.5. Future Research Directions

The findings of this review highlight several avenues for future research at the intersection of artificial intelligence and software testing. First, there is a pressing need for systematic empirical comparisons of AI algorithms applied to testing tasks. Although

numerous studies report improvements in defect prediction, test case management, and automation, the lack of standardized datasets and evaluation protocols makes it difficult to assess progress consistently. Establishing benchmarks and open repositories with shared data would enable reproducibility and facilitate meaningful comparative studies.

Second, the review shows that the interplay between problems, variables, and metrics remains fragmented. Future work should focus on integrated frameworks that jointly consider these three dimensions, since advances in one often act as enablers for the others. For example, the adoption of richer input variables has demanded new evaluation metrics, while the emergence of hybrid algorithms has shifted the way problems are addressed. Developing methodologies that explicitly link these dimensions could provide more coherent strategies for designing and assessing AI-based testing solutions.

Third, the growing application of AI in testing raises questions of interpretability, transparency, and ethical use. As models become more complex, particularly in safety-critical domains, ensuring explainability will be essential to foster trust and industrial adoption. Research should explore explainable AI techniques tailored to testing contexts, balancing predictive performance with the need for human understanding of algorithmic decisions.

Another promising line involves addressing the challenges of data scale and quality. Many of the advances reported rely on datasets of limited size or scope, which constrains the generalizability of results. Future studies should investigate mechanisms to curate high-quality, representative datasets, while also developing strategies to handle noisy, imbalanced, or incomplete data—issues that increasingly characterize industrial testing environments.

Finally, there is an opportunity to expand research toward collaborative and cross-disciplinary approaches. The integration of AI-driven testing with continuous integration pipelines, DevOps practices, and human-in-the-loop strategies could accelerate adoption in practice. Likewise, stronger collaboration between academia and industry will be critical to validate the scalability and cost-effectiveness of proposed methods.

In summary, advancing the field will require moving beyond isolated studies toward comparative, reproducible, and ethically grounded research programs. By addressing these challenges, future work can consolidate the role of AI as a transformative force in software testing, enabling more reliable, efficient, and explainable solutions for increasingly complex systems and bridging the gap between academic innovation and industrial practice.

6. Conclusions

This study proposed a comprehensive taxonomy and evolutionary analysis of AI algorithms applied to software testing, identifying the main trajectories that have shaped the field between 2014 and 2024. Beyond summarizing the classification system and evolutionary trends, this work also highlights several avenues for improvement. Future research should focus on refining the classification criteria and operational definitions of variable indicators to ensure consistency and comparability across studies. Greater emphasis should be placed on defining the semantic boundaries of categories such as test prediction, optimization, and evaluation, which remain partially overlapping in the current literature.

Additionally, the applicability of the proposed taxonomy should be extended and validated across diverse testing environments, including embedded systems, real-time software, and cloud-based testing frameworks. These contexts present different performance constraints and data characteristics, offering opportunities to assess the robustness and generalizability of AI-driven testing models.

Finally, the study encourages a stronger collaboration between academia and industry to address the gap between theoretical model design and industrial implementation. By

promoting reproducible frameworks and well-defined evaluation indicators, future studies can strengthen the reliability, interpretability, and sustainability of AI-based testing research.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/a18110717/s1, coding_book_taxonomy.xlsx: Taxonomy of AI algorithms based on software testing; PRISMA_2020_Checklist.docx: Full PRISMA 2020 checklist followed during the review.

Author Contributions: Conceptualization, A.E.-V. and D.M.; methodology, A.E.-V.; validation, A.E.-V. and D.M.; formal analysis, A.E.-V.; investigation, A.E.-V.; writing—original draft preparation, A.E.-V.; writing—review and editing, D.M.; supervision, D.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The datasets generated and analyzed during the current study are publicly available in the GitHub repository: https://github.com/escalasoft/ai-software-testing-review-data (accessed on 3 November 2025). This repository contains the filtering_articles_marked.xlsx file (article selection and screening data) and the raw_data_extraction.xlsx file (complete extracted dataset used for synthesis).

Acknowledgments: The authors would like to thank the Universidad Nacional Mayor de San Marcos (UNMSM) for supporting this research and providing access to academic resources that made this study possible.

Conflicts of Interest: The authors declare no conflict of interest.

Appendix A

Note: The identifiers [Rxx] denote the internal codes of the 66 studies included in this review. Detailed information about the algorithms, input variables, and evaluation metrics can be found in Appendices B–D. These materials, together with the complete dataset used for synthesis, are also available in the open repository: https://github.com/escalasoft/ai-software-testing-review-data.

Table A1. Selected Articles.

ID	Reference(s)	ID	Reference(s)
[R01]	R. Malhotra and K. Khan, 2024 [59]	[R02]	Z. Zulkifli et al., 2023 [60]
[R03]	F. Yang, et al., 2024 [61]	[R04]	L. Rosenbauer, et al., 2022 [62]
[R05]	A. Ghaemi and B. Arasteh, 2020 [63]	[R06]	S. Zhang et al., 2024 [64]
[R07]	M. Ali et al., 2024 [65]	[R08]	T. Rostami and S. Jalili, 2023 [66]
[R09]	M. Ali et al., 2024 [67]	[R10]	A. K. Gangwar and S. Kumar, 2024 [68]
[R11]	H. Wang et al., 2024 [69]	[R12]	G. Abaei and A. Selamat, 2015 [70]
[R13]	S. Qiu et al., 2024 [71]	[R14]	R. Sharma and A. Saha, 2018 [72]
[R15]	R. Jayanthi and M. L. Florence, 2019 [73]	[R16]	N. Nikravesh and M. R. Keyvanpour, 2024 [74]
[R17]	I. Mehmood et al., 2023 [75]	[R18]	L. Chen et al., 2018 [76]
[R19]	K. Rajnish and V. Bhattacharjee, 2022 [77]	[R20]	A. Rauf and M. Ramzan, 2018 [114]
[R21]	S. Abbas, et al., 2023 [78]	[R22]	C. Shyamala et al., 2024 [115]
[R23]	M. Bagherzadeh, et al., M., 2022 [116]	[R24]	N. A. Al-Johany et al., 2023 [79]
[R25]	Y. Lu et al., 2024 [80]	[R26]	L. Zhang and W.-T. Tsai, 2024 [81]
[R27]	W. Sun et al., 2023 [82]	[R28]	K. Pandey et al., 2020 [83]
[R29]	Z. Li et al., 2021 [84]	[R30]	P. Singh and S. Verma, 2020 [85]
[R31]	D. Manikkannan and S. Babu, 2023 [86]	[R32]	F. Tsimpourlas et al., 2022 [87]
[R33]	Y. Tang et al., 2022 [117]	[R34]	E. Sreedevi et al., 2022 [18]

Table A1. *Cont.*

ID	Reference(s)	ID	Reference(s)
[R35]	Z. Khaliq et al., 2023 [19]	[R36]	G. Kumar and V. Chopra, 2022 [88]
[R37]	M. Ma et al., 2022 [89]	[R38]	M. Sangeetha and S. Malathi, 2022 [90]
[R39]	Z. Khaliq et al., 2022 [17]	[R40]	I. Zada et al., 2024 [91]
[R41]	L. Šikić et al., 2022 [92]	[R42]	T. Hai, et al., 2022 [93]
[R43]	A. P. Widodo et al., 2023 [94]	[R44]	E. Borandag, 2023 [20]
[R45]	S. Fatima et al., 2023 [95]	[R46]	E. Borandag et al., 2019 [96]
[R47]	D. Mesquita et al., 2016 [97]	[R48]	S. Tahvili et al., 2020 [98]
[R49]	K. K. Kant Sharma et al., 2022 [99]	[R50]	B. Wójcicki and R. Dąbrowski, 2018 [100]
[R51]	F. Matloob et al., 2019 [101]	[R52]	M. Yan et al., 2020 [102]
[R53]	C. W. Yohannese et al., 2018 [103]	[R54]	L.-K. Chen et al., 2020 [104]
[R55]	B. Ma et al., 2014 [105]	[R56]	P. Singh et al., 2017 [106]
[R57]	D.-L. Miholca et al., 2018 [107]	[R58]	S. Guo et al., 2017 [108]
[R59]	L. Gonzalez-Hernandez, 2015 [109]	[R60]	M. M. Sharma et al., 2019 [110]
[R61]	G. Czibula et al., 2018 [111]	[R62]	M. Kacmajor and J. D. Kelleher, 2019 [112]
[R63]	X. Song et al., 2019 [113]	[R64]	Y. Xing et al., 2021 [118]
[R65]	A. Omer et al., 2024 [119]	[R66]	T. Shippey et al., 2019 [120]

Appendix B

Description of Algorithms.

Table A2. Selected Articles.

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R01]	2M-GWO (SVM, RF, GB, AB, KNN)	Two-Phase Modified Grey Wolf Optimizer combined with SVM (Support Vector Machine); RF (Random Forest); GB (Gradient Boosting); AB (AdaBoost); KNN (K-Nearest Neighbors) classifiers for optimization and classification	HHO, SSO, WO, JO, SCO	HHO: Harris Hawks Optimization, a metaheuristic inspired by the cooperative behavior of hawks to solve optimization problems; SSO: Social Spider Optimization, an optimization algorithm based on the communication and cooperation of social spiders; WO: Whale Optimization, an algorithm bioinspired by the hunting strategy of humpback whales; JO: Jellyfish Optimization, an optimization technique based on the movement patterns of jellyfish; SCO: Sand Cat Optimization, an algorithm inspired by the hunting strategy of desert cats to find optimal solutions.

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R02]	ANN, SVM	ANN: Artificial Neural Network, a basic neural network used for classification or regression; SVM: Support Vector Machine, a robust supervised classifier for binary classification problems	n/a	n/a
[R03]	LineFlowDP (Doc2Vec + R-GCN + GNNExplainer)	Defect prediction approach based on semantic code representation and neural graphs	CNN, DBN, BoW, Bi-LSTM, CodeT5, DeepBugs, IVDetect, LineVD, DeepLineDP, N-gram	CNN: Convolutional Neural Network, deep neural network used for automatic feature extraction in structured or unstructured data; DBN: Deep Belief Network, neural network based on layers of autoencoders to learn hierarchical data representations; BoW: Bag of Words, text or code representation model based on the frequency of appearance of words without considering the order; Bi-LSTM: Bidirectional Long Short-Term Memory, bidirectional recurrent neural network used to capture contextual information in sequences; CodeT5: Transformer Model, pre-trained transformer-based model for source code analysis and generation tasks; DeepBugs: DeepBugs Defect Detection, deep learning system designed to detect errors in source code; IVDetect: Invariant Violation Detection, a technique that seeks to detect violations of logical invariants in software programs; LineVD: Line-level Vulnerability Detector, automated system that identifies vulnerabilities in specific lines of code; DeepLineDP: Deep Line-based Defect Prediction, a deep learning-based model for predicting defects at the line of code level; N-gram: N-gram Language Model, a statistical model for processing sequences based on the frequency of occurrence of adjacent subsequences.
[R13]	CNN	Convolutional Neural Network, a neural network used for automatic feature extraction	n/a	n/a

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R22]	SDP-CMPOA (CMPOA + Bi-LSTM + Deep Maxout)	Software Defect Prediction using CMPOA optimized with Bi-LSTM and Deep Maxout activation	CNN, DBN, RNN, SVM, RF, GH + LSTM, FA, POA, PRO, AOA, COOT, BES	RNN: Recurrent Neural Network, a neural network designed to process sequential data using recurrent connections; SVM: Support Vector Machine, a robust supervised classifier for binary and multiclass classification problems; RF: Random Forest, an ensemble of decision trees used for classification and regression, robust to overfitting; GH + LSTM: Genetic Hybrid + Long Short-Term Memory, a combination of genetic optimization with an LSTM neural network to improve learning; FA: Firefly Algorithm, an optimization algorithm inspired by the luminous behavior of fireflies to solve complex problems; POA: Pelican Optimization Algorithm, an optimization technique based on the collective behavior of pelicans; PRO: Progressive Optimization, an optimization approach that iteratively adjusts parameters to improve results; AOA: Arithmetic Optimization Algorithm, a metaheuristic based on arithmetic operations to explore and exploit the search space; COOT: Coot Bird Optimization, an optimization algorithm inspired by the movements of coot-type aquatic birds; BES: Bacterial Foraging Optimization, a metaheuristic inspired by the foraging strategy of bacteria.
[R24]	DT, NB, RF, LSVM	DT: Decision Tree, classifier based on decision trees, NB: Naïve Bayes, probabilistic classifier based on Bayes theory, RF: Random Forest, ensemble of decision trees for classification and regression, LSVM: Linear Support Vector Machine, linear version of SVM	n/a	n/a
[R10]	PoPL(Hybrid)	Paired Learner Approach, a hybrid technique for handling concept drift in defect prediction	n/a	n/a

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R11]	bGWO (ANN, DT, KNN, NB, SVM)	Binary Grey Wolf Optimizer combined with multiple classifiers	ACO	Ant Colony Optimization, a metaheuristic technique based on the collective behavior of ants to solve route optimization or combinatorial problems
[R12]	FMR, FMRT	Fuzzy Min-Max Regression and its variant for prediction	NB, RF, ACN, ACF	NB: Naïve Bayes, a simple probabilistic classifier based on the application of Bayes' theorem with independence between attributes; ACN: Artificial Cognitive Network, an artificial network model inspired by cognitive systems for classification or pattern analysis; ACF: Artificial Cooperative Framework, an artificial cooperative framework designed to improve accuracy in prediction or classification tasks.
[R15]	LM, BP, BR, BR + NN	LM: Linear Model, linear regression model, BP: Backpropagation, training algorithm for neural networks, BR: Bayesian Regularization, technique to avoid overfitting in neural networks, BR + NN: Bayesian Regularized Neural Network, Bayesian regularized neural network	SVM, DT, KNN, NN	DT: Decision Tree, a classification or regression model based on a decision tree structure; KNN: K-Nearest Neighbors, a classifier based on the similarity between instances in the feature space; NN: Neural Network, an artificial neural network used for supervised or unsupervised learning in various tasks.
[R16]	DEPT-C, DEPT-M1, DEPT-M2, DEPT-D1, DEPT-D2	Variants of a specific DEPT approach to prioritization or prediction in software testing	DE, GS, RS	DE: Differential Evolution, an evolutionary optimization algorithm used to solve continuous and nonlinear problems; GS: Grid Search, a systematic search method for hyperparameter optimization in machine learning models; RS: Random Search, a hyperparameter optimization technique based on the random selection of combinations.
[R42]	MLP	Multilayer Perceptron, a neural network with multiple hidden layers.	n/a	
[R18]	C4.5 +ADB	C4.5 Decision Tree Algorithm Combined with AdaBoost to Improve Accuracy.	ERUS, NB, NB + Log, RF, DNC, SMT + NB, RUS + NB, SMTBoost, RUSBoost	ERUS: Ensemble Random Under Sampling, class balancing method based on combined random undersampling in ensemble;

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R18]	C4.5 +ADB	C4.5 Decision Tree Algorithm Combined with AdaBoost to Improve Accuracy.	ERUS, NB, NB + Log, RF, DNC, SMT + NB, RUS + NB, SMTBoost, RUSBoost	NB + Log: Naïve Bayes + Logistic Regression, hybrid approach that combines Naïve Bayes probabilities with a logistic classifier; DNC: Dynamic Nearest Centroid, classifier based on dynamic centroids to improve accuracy; SMT + NB: Synthetic Minority Technique + Naïve Bayes, combination of class balancing with Bayesian classification; RUS + NB: Random Under Sampling + Naïve Bayes, majority class reduction technique combined with Naïve Bayes; SMTBoost: Synthetic Minority Oversampling Technique Boosting, balancing method combined with boosting to improve classification; RUSBoost: Random Under Sampling Boosting, ensemble method based on undersampling and boosting to improve prediction.
[R28]	KPCA + ELM	Kernel Principal Component Analysis combined with Extreme Learning Machine	SVM, NB, LR, MLP, PCA + ELM	LR: Logistic Regression, a statistical model used for binary classification using the sigmoid function; MLP: Multilayer Perceptron, an artificial neural network with one or more hidden layers for classification or regression; PCA + ELM: Principal Component Analysis + Extreme Learning Machine, a hybrid approach that reduces dimensionality and applies ELM for classification.
[R47]	rejoELM, IrejoELM	Improved variants of the Extreme Learning Machine applying its own techniques.	rejoNB, rejoRBF	rejoNB: Re-joined Naïve Bayes, an improved variant of Naïve Bayes for classification; rejoRBF: Re-joined Radial Basis Function, a variant based on RBF for classification or regression tasks.
[R29]	WPA-PSO + DNN, WPA-PSO + self-encoding	Whale + Particle Swarm Optimization combined with Deep Neural Networks or Autoencoders.	Grid, Random, PSO, WPA	Grid: Grid Search, an exhaustive search technique for hyperparameter optimization; Random: Random Search, a random parameter optimization strategy; PSO: Particle Swarm Optimization, an optimization algorithm inspired by the behavior of particle swarms; WPA: Whale Particle Algorithm, a metaheuristic that combines whale and particle optimization strategies.

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R30]	ACO	Ant Colony Optimization, a technique inspired by ant behavior for optimization.	NB, J48, RF	J48: J48 Decision Tree, implementation of the C4.5 algorithm in WEKA software for classification.
[R41]	DP + GCNN	Defect Prediction using Graph Convolutional Neural Network	LRC, RFC, DBN, CNN, SEML, MPT, DP-T, CSEM	LRC: Logistic Regression Classifier, a variant of logistic regression applied to classification tasks; RFC: Random Forest Classifier, an ensemble of decision trees for robust classification; SEML: Software Engineering Machine Learning, an approach that applies machine learning techniques to software engineering; MPT: Modified Particle Tree, a tree-based algorithm for optimization; DP-T: Defect Prediction-Tree, a tree-based approach for defect prediction; CSEM: Code Structural Embedding Model, a model that uses structural code embeddings for prediction or classification.
[R44]	RNNBDL	Recurrent Neural Network with Bayesian Deep Learning	LSTM, BiLSTM, CNN, SVM, NB, KNN, KStar, Random Tree	LSTM: Long Short-Term Memory, a recurrent neural network specialized in learning long-term dependencies in sequences; BiLSTM: Bidirectional Long Short-Term Memory, a bidirectional version of LSTM that captures past and future context in sequences; KStar: KStar Instance-Based Classifier, a nearest-neighbor classifier with a distance function based on transformations; Random Tree: Random Tree Classifier, a classifier based on randomly generated decision trees.
[R50]	Naïve Bayes (GaussianNB)	Naïve Bayes variant using Gaussian distribution	n/a	n/a
[R51]	Stacking + MLP (J48, RF, SMO, IBK, BN) + BF, GS, GA, PSO, RS, LFS	Stacking ensemble of multiple classifiers and meta-heuristics	n/a	n/a
[R53]	TS-ELA (ELA + IG + SMOTE + INFFC) + (BaG, RaF, AdB, LtB, MtB, RaB, StK, StC, VoT, DaG, DeC, GrD, RoF)	Hybrid technique that combines multiple balancing, selection and induction techniques	DTa, DSt	DTa: Decision Tree (Adaptive), a variant of the adaptive decision tree for classification; DSt: Decision Stump, a single-split decision tree, used in ensemble methods.

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R55]	CBA2	Classification Based on Associations version 2	C4.5, CART, ADT, RIPPER, DT	C4.5: C4.5 Decision Tree, a classic decision tree algorithm used in classification; CART: Classification and Regression Tree, a tree technique for classification or regression tasks; ADT: Alternating Decision Tree, a tree-based algorithm with alternating prediction and decision nodes; RIPPER: Repeated Incremental Pruning to Produce Error Reduction, a rule-based algorithm for classification.
[R57]	HyGRAR (MLP, RBFN, GRANUM)	Hybrid of MLP, radial basis networks and GRAR algorithm for classification.	SOM, KMeans-QT, XMeans, EM, GP, MLR, BLR, LR, ANN, SVM, CCN, GMDH, GEP, SCART, FDT-O, FDT-E, DT-Weka, BayesNet, MLP, RBFN, ADTree, DTbl, CODEP-Log, CODEP-Bayes	SOM: Self-Organizing Map, unsupervised neural network used for clustering and data visualization; KMeans-QT: K-Means Quality Threshold, a variant of the K-Means algorithm with quality thresholds for clusters; XMeans: Extended K-Means, an extended version of K-Means that automatically optimizes the number of clusters; EM: Expectation Maximization, an iterative statistical technique for parameter estimation in mixture models; GP: Genetic Programming, an evolutionary programming technique for solving optimization or learning problems; MLR: Multiple Linear Regression, a statistical model for predicting a continuous variable using multiple predictors; BLR: Bayesian Linear Regression, a linear regression under a Bayesian approach to incorporate uncertainty; ANN: Artificial Neural Network, an artificial neural network used in classification, regression, or prediction tasks; CCN: Convolutional Capsule Network, a convolutional capsule network for pattern recognition; GMDH: Group Method of Data Handling, a technique based on polynomial networks for predictive modeling; GEP: Gene Expression Programming, an evolutionary technique based on genetic programming for symbolic modeling; SCART: Soft Classification and Regression Tree, a decision tree variant that allows fuzzy or soft classification; FDT-O: Fuzzy Decision Tree-Option, a decision tree variant with the incorporation of fuzzy logic;

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R57]	HyGRAR (MLP, RBFN, GRANUM)	Hybrid of MLP, radial basis networks and GRAR algorithm for classification.	SOM, KMeans-QT, XMeans, EM, GP, MLR, BLR, LR, ANN, SVM, CCN, GMDH, GEP, SCART, FDT-O, FDT-E, DT-Weka, BayesNet, MLP, RBFN, ADTree, DTbl, CODEP-Log, CODEP-Bayes	FDT-E: Fuzzy Decision Tree-Enhanced, an improved version of fuzzy decision trees; DT-Weka: Decision Tree Weka, an implementation of decision trees within the WEKA platform; BayesNet: Bayesian Network, a probabilistic classifier based on Bayesian networks; RBFN: Radial Basis Function Network, a neural network based on radial basis functions for classification or regression; ADTree: Alternating Decision Tree, a technique based on alternating decision and prediction trees; DTbl: Decision Table, a simple classifier based on decision tables; CODEP-Log: Code Execution Prediction-Logistic Regression, a defect prediction approach using logistic regression; CODEP-Bayes: Code Execution Prediction-Naïve Bayes, a prediction approach based on Naïve Bayes.
[R65]	ME-SFP + [DT], ME-SFP + [MLP]	Multiple Ensemble with Selective Feature Pruning with base classifiers.	Bagging + DT, Bagging + MLP, Boosting + DT, Boosting + MLP, Stacking + DT, Stacking + MLP, Indi + DT, Indi + MLP, Classic + ME	Bagging + DT: Bootstrap Aggregating + Decision Tree, an ensemble method that uses decision trees to improve accuracy; Bagging + MLP: Bagging + Multilayer Perceptron, an ensemble method that applies MLP networks; Boosting + DT: Boosting + Decision Tree, an ensemble method where the weak classifiers are decision trees; Boosting + MLP: Boosting + MLP, a combination of boosting and MLP neural networks; Stacking + DT: Stacking + Decision Tree, a stacked ensemble that uses decision trees; Stacking + MLP: Stacking + MLP, a stacked ensemble with MLP networks; Indi + DT: Individual Decision Tree, an approach based on individual decision trees within a comparison or ensemble scheme; Indi + MLP: Individual MLP, an MLP neural network used independently in experiments or ensembles; Classic + ME: Classic Multiple Ensemble, a classic configuration of ensemble methods.

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R66]	AST n-gram + J48, AST n-gram + Logistic, AST n-gram + Naive Bayes	Approach based on AST n-gram feature extraction combined with different classifiers	n/a	n/a
[R07]	IECGA (RF + SVM + NB + GA)	Improved Evolutionary Cooperative Genetic Algorithm with Multiple Classifiers	RF, SVM, NB	NB: Naïve Bayes, simple probabilistic classifier based on Bayes theory.
[R09]	VESDP (RF + SVM + NB + ANN)	Variant Ensemble Software Defect Prediction	RF, SVM, NB, ANN	ANN: Artificial Neural Network, artificial neural network used in classification or regression tasks
[R17]	MLP, BN, Lazy IBK, Rule ZeroR, J48, LR, RF, DStump, SVM	BN: Bayesian Network, classifier based on Bayesian networks, Lazy IBK: Instance-Based K Nearest Neighbors, Rule ZeroR: Trivial classifier without predictor variables, J48: Implementation of C4.5 in WEKA, LR: Logistic Regression, logistic regression, DStump: Decision Stump, decision tree of depth 1	n/a	n/a
[R19]	CONVSDP (CNN), DNNSDP (DNN)	Convolutional Neural Network applied to defect prediction., Deep Neural Network applied to defect prediction	RF, DT, NB, SVM	RF: Random Forest, an ensemble of decision trees that improves accuracy and overfitting control.
[R21]	ISDPS (NB + SVM + DT)	Intelligent Software Defect Prediction System combining classifiers	NB, SVM, DT, Bagging, Vouting, Stacking	Bagging: Bootstrap Aggregating, an ensemble technique that improves the stability of classifiers; Vouting: Voting Ensemble, an ensemble method that combines the predictions of multiple classifiers using voting; Stacking: Stacked Generalization, an ensemble technique that combines multiple models using a meta-classifier.
[R33]	2SSEBA (2SSSA, ELM, Bagging Ensemble)	Two-Stage Salp Swarm Algorithm + ELM with Ensemble	ELM, SSA + ELM, 2SSSA + ELM, KPWE, SEBA	ELM: Extreme Learning Machine, a single-layer, fast-learning neural network. SSA + ELM: Salp Swarm Algorithm + ELM, a combination of the bio-inspired SSA algorithm and ELM;

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R33]	2SSEBA (2SSSA, ELM, Bagging Ensemble)	Two-Stage Salp Swarm Algorithm + ELM with Ensemble	ELM, SSA + ELM, 2SSSA + ELM, KPWE, SEBA	2SSSA + ELM: Two-Stage Salp Swarm Algorithm + ELM, an improved version of the SSA approach combined with ELM; KPWE: Kernel Principal Wavelet Ensemble, a method that combines wavelet transforms with kernel techniques for classification; SEBA: Swarm Enhanced Bagging Algorithm, an enhanced ensemble technique using swarm algorithms
[R38]	MODL-SBP (CNN-BiLSTM + CQGOA)	Hybrid model combining CNN, BiLSTM and CQGOA optimization	SVM-RBF, KNN + EM, NB, DT, LDA, AdaBoost,	SVM-RBF: Support Vector Machine with Radial Basis Function, an SVM using RBF kernels for nonlinear separation; KNN + EM: K-Nearest Neighbors + Expectation Maximization, a combination of KNN classification with an EM algorithm for clustering or imputation; LDA: Linear Discriminant Analysis, a statistical technique for dimensionality reduction and classification; AdaBoost: Adaptive Boosting, an ensemble technique that combines weak classifiers to improve accuracy
[R46]	MVFS (MVFS + NB, MVFS + J48, MVFS + IBK)	Multiple View Feature Selection applied to different classifiers	IG, CO, RF, SY	IG: Information Gain, a statistical measure used to select attributes in decision models; CO: Cut-off Optimization, a technique that adjusts cutoff points in classification models; SY: Symbolic Learning, a symbolic learning-based approach for classification or pattern discovery tasks.
[R06]	HFEDL (CNN, BiLSTM + Attention)	Hierarchical Feature Ensemble Deep Learning	n/a	n/a
[R40]	KELM + WSO	Kernel Extreme Learning Machine combined with Weight Swarm Optimization	SNB, FLDA, GA + DT, CGenProg	SNB: Selective Naïve Bayes, an improved version of Naïve Bayes based on the selection of relevant attributes; FLDA: Fisher Linear Discriminant Analysis, a dimensionality reduction technique optimized for class separation; GA + DT: Genetic Algorithm + Decision Tree, a combination of genetic algorithms with decision trees for parameter selection or optimization;

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R40]	KELM + WSO	Kernel Extreme Learning Machine combined with Weight Swarm Optimization	SNB, FLDA, GA + DT, CGenProg	CGenProg: Code Genetic Programming, a genetic programming application for automatic code improvement or repair.
[R49]	CCFT + CNN	Combination of Code Feature Transformation + CNN	RF, DBN, CNN, RNN, CBIL, SMO	CBIL: Classifier Based Incremental Learning, an incremental approach to supervised learning based on classifiers; SMO: Sequential Minimal Optimization, an efficient algorithm for training SVMs
[R58]	KTC (IDR + NB, IDR + SVM, IDR + KNN, IDR + J48)	Keyword Token Clustering combined with different classifiers	NB, KNN, SVM, J48	Set of standard classifiers (Naïve Bayes, K-Nearest Neighbors, Support Vector Machine, J48 Decision Tree) applied in various classification tasks.
[R45]	Flakify (CodeBERT)	CodeBERT-based model for unstable test detection	FlakeFlagger	FlakeFlagger: Flaky Test Flagging Model, a model designed to identify unstable tests or flakiness in software testing.
[R34]	SVM + MLP + RF	SVM: Support Vector Machine + MLP: Multilayer Perceptron + RF: Random Forest, hybrid ensemble that combines SVM, MLP neural networks and Random Forest to improve accuracy.	SVM, ANN, RF	SVM: Support Vector Machine, a robust classifier widely used for supervised classification problems; ANN: Artificial Neural Network, an artificial neural network for classification, regression, or prediction tasks; RF: Random Forest, an ensemble technique based on multiple decision trees to improve accuracy and robustness.
[R56]	FRBS	Fuzzy Rule-Based System, a system based on fuzzy rules used for classification or decision making	C4.5, RF, NB	C4.5: Decision Tree, a classic decision tree algorithm used for classification; NB: Naïve Bayes, a simple probabilistic classifier based on the application of Bayes' theorem.
[R04]	XCSF-ER	Extended Classifier System with Function Approximation-Enhanced Rule, extended rule-based system with approximation and enhancement capabilities	ANN, RS, XCSF	RS: Random Search, a hyperparameter optimization technique based on random selection; XCSF: Extended Classifier System with Function Approximation, a rule-based evolutionary learning system.
[R60]	KNN	K-Nearest Neighbors, a classifier based on the similarity between nearby instances in the feature space	LR, LDA, CART, NB, SVM	LR: Logistic Regression, a statistical model for binary or multiclass classification; LDA: Linear Discriminant Analysis, a method for dimensionality reduction and supervised classification;

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R60]	KNN	K-Nearest Neighbors, a classifier based on the similarity between nearby instances in the feature space	LR, LDA, CART, NB, SVM	CART: Classification and Regression Trees, a tree technique used in classification and regression.
[R64]	AFSA	Artificial Fish Swarm Algorithm, a bio-inspired metaheuristic based on fish swarm behavior for optimization	GA, K-means Clustering, NSGA-II, IA	GA: Genetic Algorithm, an evolutionary algorithm based on natural selection for solving complex problems; K-means Clustering: K-means Clustering Algorithm, an unsupervised technique for grouping data into distance-based clusters; NSGA-II: Non-dominated Sorting Genetic Algorithm II, a widely used multi-objective evolutionary algorithm; IA: Intelligent Agent, a computational system that perceives its environment and makes autonomous decisions.
[R35]	T5 (YOLOv5)	Text-to-Text Transfer Transformer + You Only Look Once v5, combining language processing with object detection in images	n/a	
[R39]	EfficientDet, DETR, T5, GPT-2	EfficientDet: EfficientDet Object Detector, a deep learning model optimized for object detection in images; DETR: Detection Transformer, a transformer-based model for object detection in computer vision; T5: Text-to-Text Transfer Transformer, a deep learning model for translation, summarization, and other NLP tasks; GPT-2: Generative Pre-trained Transformer 2, a transformer-based autoregressive language model.	n/a	

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R14]	MFO	Moth Flame Optimization, a bio-inspired optimization algorithm based on the behavior of moths around flames	FA, ACO	FA: Firefly Algorithm, a metaheuristic inspired by the light behavior of fireflies; ACO: Ant Colony Optimization, a bio-inspired metaheuristic based on cooperative pathfinding in ants.
[R48]	IFROWANN av-w_1	Improved Fuzzy Rough Weighted Artificial Neural Network, a neural network with fuzzy weighting and approximation	EUSBoost, SMOTE + C4.5, CS + SVM, CS + C4.5	EUSBoost: Evolutionary Undersampling Boosting, an ensemble technique that balances classes using evolutionary undersampling; SMOTE + C4.5: Synthetic Minority Oversampling + C4.5, a hybrid technique for class balancing and classification; CS + SVM: Cost-Sensitive SVM, a cost-sensitive version of the SVM classifier; CS + C4.5: Cost-Sensitive C4.5, a cost-sensitive version applied to C4.5 trees.
[R32]	NN (LSTM + MLP)	Neural Network (LSTM + Multilayer Perceptron), a hybrid neural network that combines LSTM and MLP networks	Hierarchical Clustering	Hierarchical Clustering Algorithm, an unsupervised technique that groups data hierarchically.
[R43]	EfficientNet-B1	EfficientNet-B1, a convolutional neural network optimized for image classification with high efficiency	CNN, VGG-16, ResNet-50, MobileNet-V3	CNN: Convolutional Neural Network, a deep neural network used for automatic feature extraction in images, text, or structured data; VGG-16: Visual Geometry Group 16-layer CNN, a deep convolutional network architecture with 16 layers designed for image classification tasks; ResNet-50: Residual Neural Network 50 layers, a convolutional neural network with residual connections that facilitate the training of deep networks; MobileNet-V3: MobileNet Version 3, a lightweight convolutional network architecture optimized for mobile devices and computer vision tasks with low resource demands.
[R62]	NMT	Neural Machine Translation, a neural network-based system for automatic language translation	n/a	

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R23]	RL-based-CI	Reinforcement Learning–based Continuous Integration, a learning-driven approach that leverages reinforcement learning agents to optimize the scheduling, selection, or prioritization of test cases and builds in continuous integration pipelines. It continuously adjusts decisions based on rewards obtained from build outcomes or defect detection performance.	RL-BS1,RL-BS2	Reinforcement Learning–based Baseline Strategies 1 and 2, two baseline configurations designed to benchmark the performance of RL-based continuous integration systems. RL-BS1 generally employs static reward structures or fixed exploration parameters, while RL-BS2 integrates adaptive reward tuning and dynamic exploration policies to enhance decision-making efficiency in CI environments.
[R36]	ACO + NSA	Ant Colony Optimization + Negative Selection Algorithm, a combination of ant-based optimization and immune-inspired negative selection algorithm	Random Testing, ACO, NSA	Random Testing: A software testing technique that randomly generates inputs to uncover errors; NSA: Negative Selection Algorithm, a bio-inspired algorithm based on the immune system used to detect anomalies or intrusions.
[R05]	SFLA	Shuffled Frog-Leaping Algorithm, a metaheuristic algorithm based on the social behavior of frogs to solve complex problems	GA, PSO, ACO, ABC, SA	GA: Genetic Algorithm, an evolutionary algorithm based on principles of natural selection for solving complex optimization problems; PSO: Particle Swarm Optimization, an optimization algorithm inspired by swarm behavior for finding optimal solutions; ABC: Artificial Bee Colony, an optimization algorithm bioinspired by bee behavior for finding solutions; SA: Simulated Annealing, a probabilistic optimization technique based on the physical annealing process of materials.
[R26]	ERINet	Enhanced Residual Inception Network, improved neural architecture for complex pattern recognition	SIFT, SURF, ORB	SIFT: Scale-Invariant Feature Transform, a computer vision algorithm for keypoint detection and description in images; SURF: Speeded-Up Robust Features, a fast and robust algorithm for local feature detection in images;

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R26]	ERINet	Enhanced Residual Inception Network, improved neural architecture for complex pattern recognition	SIFT, SURF, ORB	ORB: Oriented FAST and Rotated BRIEF, an efficient method for visual feature detection and image matching.
[R63]	ER -Fuzz (Word2Vec + LSTM)	Error-Revealing Fuzzing with Word2Vec and LSTM, a hybrid approach for generating and analyzing fault-causing inputs	AFL, AFLFast, DT, LSTM	AFL: American Fuzzy Lop, a fuzz testing tool used to discover vulnerabilities by automatically generating malicious input; AFLFast: American Fuzzy Lop Fast, an optimized version of AFL that improves the speed and efficiency of bug detection through fuzzing; DT: Decision Tree, a classifier based on a hierarchical decision structure for classification or regression tasks; LSTM: Long Short-Term Memory, a recurrent neural network designed to learn long-term dependencies in sequences.
[R27]	HashC-NC	Hash Coverage-Neuron Coverage, a test coverage approach based on neuron activation in deep networks	NC, 2-way, 3-way, INC, SC, KMNC, HashC-KMNC, TKNC	(Evaluation criteria) NC, 2-way, 3-way, INC, SC, KMNC, HashC-KMNC, TKNC: Set of metrics or techniques for evaluating coverage and diversity in software testing based on neuron activation, combinatorics and structural coverage.
[R20]	NSGA-II, MOPSO	NSGA-II: Non-dominated Sorting Genetic Algorithm II, a multi-objective evolutionary algorithm widely used in optimization; MOPSO: Multi-Objective Particle Swarm Optimization, a multi-objective version of particle swarm optimization	Single-objective GA, PSO	Single-objective GA: Single-Objective Genetic Algorithm, a classic genetic algorithm focused on optimizing a single specific objective
[R37]	CVDF DYNAMIC (Bi-LSTM + GA)	Cross-Validation Dynamic Feature Selection using Bi-LSTM and Genetic Algorithm for adaptive feature selection	NeuFuzz, VDiscover, AFLFast	NeuFuzz: Neural Fuzzing System, a deep learning-based system for automated test data generation; VDiscover: Vulnerability Discoverer, an automated vulnerability detection tool using dynamic or static analysis;

Table A2. *Cont.*

ID	Novel Algorithm(s)	Description	Existing Algorithm(s)	Description
[R37]	CVDF DYNAMIC (Bi-LSTM + GA)	Cross-Validation Dynamic Feature Selection using Bi-LSTM and Genetic Algorithm for adaptive feature selection	NeuFuzz, VDiscover, AFLFast	AFLFast: American Fuzzy Lop Fast, a (repeated) optimized system for efficient fuzz testing.
[R52]	ARTDL	Adaptive Random Testing Deep Learning, a software testing approach that combines adaptive sampling techniques with deep learning models	RT	RT: Random Testing, a basic strategy for generating random data for software testing
[R25]	MTUL (Autoencoder)	Autoencoder-based Multi-Task Unsupervised Learning, used for unsupervised learning and anomaly detection	n/a	
[R61]	RL	Reinforcement Learning, a reward-based machine learning technique for sequential decision-making	GA, ACO, RS	GA: Genetic Algorithm, ACO: Ant Colony Optimization and RS: Random Search, metaheuristics or search strategies combined or applied individually for optimization or classification.
[R08]	FrMi	Fractional Minkowski Distance, an improved distance metric for distance-based classifiers	SVM, RF, DT, LR, NB, CNN	Set of traditional classifiers SVM: Support Vector Machine, RF: Random Forest, DT: Decision Tree, LR: Logistic Regression, NB: Naïve Bayes, CNN: Convolutional Neural Network, applied to different prediction or classification tasks.
[R31]	MLP	Multilayer Perceptron, a neural network with multiple hidden layers widely used in classification.	Random Strategy, Total Strategy, Additional Strategy	Test case selection or prioritization strategies based on random, exhaustive, or incremental criteria.
[R54]	LSTM	Long Short-Term Memory, a recurrent neural network specialized in learning long-term temporal dependencies	n/a	
[R59]	MiTS	Minimal Test Suite, an approach for automatically generating a minimal set of test cases	n/a	

Appendix C

Variables used in AI studies for ST.

Table A3. Description of variables.

Subcategory	Variable	Description	Study ID
Source Code Structures	LOC	Total lines of source code	[R11], [R12], [R15], [R22], [R16], [R18], [R28], [R47], [R44], [R51], [R55], [R65], [R07], [R09], [R17], [R46], [R40], [R66], [R34], [R56], [R64], [R42], [R13], [R10], [R19], [R06]
Source Code Structures	v(g)	Cyclomatic complexity of the control graph	[R11], [R12], [R15], [R18], [R28], [R29], [R30], [R44], [R51], [R55], [R46], [R40], [R56], [R36], [R05], [R42], [R10], [R06]
Source Code Structures	eV(g)	Essential complexity (EVG)	[R11], [R12], [R15], [R18], [R28], [R29], [R44], [R46], [R40], [R56]
Source Code Structures	iv(g)	Information Flow Complexity (IVG)	[R11], [R15], [R18], [R28], [R29], [R30], [R44], [R40], [R56]
Source Code Structures	npm	Number of public methods	[R01], [R16], [R28], [R65], [R49], [R34]
Source Code Structures	NOM	Total number of methods	[R47], [R46], [R06]
Source Code Structures	NOPM	Number of public methods	[R47], [R46]
Source Code Structures	NOPRM	Number of protected methods	[R47], [R46]
Source Code Structures	NOMI	Number of internal or private methods	[R01], [R47], [R46]
Source Code Structures	Loc_com	Lines of code that contain comments	[R01], [R15], [R11], [R28], [R29], [R44], [R50], [R51], [R21], [R46], [R66], [R56]
Source Code Structures	Loc_blank	Blank lines in the source file	[R01], [R11], [R15], [R28], [R29], [R30], [R50], [R51], [R21], [R46], [R34], [R56]
Source Code Structures	Loc_executable	Lines containing executable code	[R01], [R28], [R51], [R07], [R34], [R56]
Source Code Structures	LOCphy	Total physical lines of source code	[R29], [R41]
Source Code Structures	CountLineCodeDecl	Lines dedicated to declarations	[R01]
Source Code Structures	CountLineCode	Total lines of code without comments	[R01], [R28], [R44], [R46], [R49], [R45]
Source Code Structures	Locomment	Number of lines containing only comments	[R15], [R22], [R28], [R29], [R44], [R50], [R51], [R09], [R46], [R66], [R34]
Source Code Structures	Branchcount	Total number of conditional branches (if, switch, etc.)	[R15], [R30], [R50], [R51], [R07], [R46], [R34], [R56], [R19]
Source Code Structures	Avg_CC	Average cyclomatic complexity of the methods	[R28], [R65], [R34]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Source Code Structures	max_cc	Maximum cyclomatic complexity of all methods	[R16], [R28], [R30], [R07], [R34]
Source Code Structures	NOA	Total number of attributes in a class	[R47], [R46]
Source Code Structures	NOPA	Number of public attributes	[R47], [R46]
Source Code Structures	NOPRA	Number of protected attributes	[R47], [R46]
Source Code Structures	NOAI	Number of internal/private attributes	[R47], [R46]
Source Code Structures	NLoops	Total number of loops (for, while)	[R29]
Source Code Structures	NLoopsD	Number of nested loops	[R29]
Source Code Structures	max_cc	Maximum observed cyclomatic complexity between methods	[R50], [R51], [R65], [R17]
Source Code Structures	CALL_PAIRS	Number of pairs of calls between functions	[R51], [R09], [R56]
Source Code Structures	CONDITION_COUNT	Number of boolean conditions (if, while, etc.)	[R51], [R56]
Source Code Structures	CYCLOMATI C_DENSITY (vd(G))	Cyclomatic complexity density relative to code size	[R51], [R21], [R56]
Source Code Structures	DECISION_count	Number of decision points	[R51], [R56]
Source Code Structures	DECISION_density (dd(G))	Proportion of decisions to total code	[R51], [R56]
Source Code Structures	EDGE_COUNT	Number of edges in the control flow graph	[R51], [R56]
Source Code Structures	ESSENTIAL_COMPLEXITY (ev(G))	Unstructured part of the control flow (minimal structuring)	[R51], [R40], [R34], [R56]
Source Code Structures	ESSENTIAL_DENSITY (ed(G))	Density of the essence complexity	[R51], [R56]
Source Code Structures	PARAMETER_COUNT	Number of parameters used in functions or methods	[R51], [R21], [R56], [R02]
Source Code Structures	MODIFIED_CONDITION _COUNT	Counting modified conditions (e.g., if, while)	[R51], [R56]
Source Code Structures	MULTIPLE_CONDITION _COUNT	Counting compound decisions (e.g., if (a && b))	[R51], [R56]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Source Code Structures	NODE_COUNT	Total number of nodes in the control graph	[R51], [R56]
Source Code Structures	NORMALIZED_CYLOMATIC_COMP (Normv(G))	Cyclomatic complexity divided by lines of code	[R51], [R56]
Source Code Structures	NUMBER_OF_LINES	Total number of lines in the source file	[R51], [R56]
Source Code Structures	PERCENT_COMMENTS	Percentage of lines that are comments	[R51], [R17], [R21], [R56]
Halstead Metrics	n1, n2/N1, N2	Number of operators (n1) and unique operands (n2)	[R24], [R50], [R56]
Halstead Metrics	V	Program volume	[R11], [R24], [R15], [R29], [R50], [R55], [R46], [R66], [R56]
Halstead Metrics	L	Expected program length	[R11], [R24], [R15], [R44], [R51], [R53], [R55], [R46], [R66], [R56]
Halstead Metrics	D	Code difficulty	[R11], [R24], [R15], [R29], [R46], [R66], [R56]
Halstead Metrics	E	Implementation effort	[R11], [R24], [R15], [R46], [R66], [R56]
Halstead Metrics	N	Total length: sum of operators and operands	[R15], [R29], [R50], [R46], [R66], [R53], [R57], [R11], [R12], [R18], [R66], [R34]
Halstead Metrics	B	Estimated number of errors	[R15], [R46], [R66], [R56]
Halstead Metrics	I	Required intelligence level	[R11], [R15], [R29], [R46], [R56], [R56]
Halstead Metrics	T	Estimated time to program the software	[R11], [R15], [R29], [R46], [R56]
Halstead Metrics	uniq_Op	Number of unique operators	[R11], [R12], [R15], [R28], [R29], [R51], [R53], [R57], [R46], [R34], [R19]
Halstead Metrics	uniq_Opnd	Number of unique operators	[R11], [R12], [R15], [R28], [R29], [R51], [R53], [R57], [R46], [R34], [R19]
Halstead Metrics	total_Op	Total operators used	[R11], [R15], [R28], [R29], [R30], [R51], [R53], [R55], [R21], [R46]
Halstead Metrics	total opnd	Total operands used	[R15], [R28], [R29], [R51], [R53], [R55], [R46], [R66]
Halstead Metrics	hc	Halstead Complexity (may be variant specific)	[R28]
Halstead Metrics	hd	Halstead Difficulty	[R28]
Halstead Metrics	he	Halstead Effort	[R28], [R30], [R51], [R07], [R34]
Halstead Metrics	hee	Halstead Estimated Errors	[R28], [R51], [R53], [R34]
Halstead Metrics	hl	Halstead Length	[R28], [R51], [R34]
Halstead Metrics	hlen	Estimated Halstead Length	[R28], [R09]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Halstead Metrics	hpt	Halstead Programming Time	[R28], [R51]
Halstead Metrics	hv	Halstead Volume	[R28], [R51], [R34]
Halstead Metrics	Lv	Logical level of program complexity	[R29], [R34]
Halstead Metrics	HALSTEAD_CONTENT	Content calculated according to the Halstead model	[R51], [R21], [R34]
Halstead Metrics	HALSTEAD_DIFFICULTY	Estimated difficulty of understanding the code	[R51], [R34]
OO Metrics	amc	Average Method Complexity	[R16], [R28], [R65], [R33], [R38], [R34]
OO Metrics	ca	Afferent coupling: number of classes that depend on this	[R16], [R28], [R65], [R49]
OO Metrics	cam	Cohesion between class methods	[R16], [R28], [R65], [R17]
OO Metrics	cbm	Coupling between class methods	[R16], [R28], [R65], [R49], [R34]
OO Metrics	cbo	Coupling Between Object classes	[R16], [R28], [R47], [R57], [R65], [R46], [R49], [R34]
OO Metrics	dam	Data Access Metric	[R16], [R28], [R65], [R49], [R34]
OO Metrics	dit	Depth of Inheritance Tree	[R16], [R28], [R47], [R65], [R46], [R49], [R34]
OO Metrics	ic	Inheritance Coupling	[R16], [R28], [R65], [R49], [R34]
OO Metrics	lcom	Lack of Cohesion of Methods	[R16], [R28], [R47], [R65], [R17], [R46], [R49], [R34]
OO Metrics	lcom3	Improved variant of LCOM for detecting cohesion	[R16], [R28], [R65], [R34]
OO Metrics	mfa	Measure of Functional Abstraction	[R16], [R28], [R65], [R34]
OO Metrics	moa	Measure of Aggregation	[R16], [R28], [R65], [R34]
OO Metrics	noc	Number of Children: number of derived classes	[R16], [R28], [R47], [R17], [R46], [R34]
OO Metrics	wmc	Weighted Methods per Class	[R16], [R28], [R47], [R57], [R65], [R46], [R34]
OO Metrics	FanIn	Number of functions or classes that call a given function	[R47], [R29], [R44], [R46]
OO Metrics	FanOut	Number of functions called by a given function	[R47], [R29], [R44], [R46]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Software Quality Metrics	rfc	Fan-in OO: Classes that call this class	[R01], [R16], [R28], [R47], [R57], [R46], [R66], [R34]
Software Quality Metrics	ce	OO Fan-out: Classes that this class uses	[R01], [R16], [R28], [R65], [R49], [R34]
Software Quality Metrics	DESIGN_COMPLEXITY (iv(G))	Composite measure of design complexity	[R51], [R09], [R40], [R34], [R56]
Software Quality Metrics	DESIGN_DENSITY (id(G))	Density of design elements per code unit	[R51], [R56]
Software Quality Metrics	GLOBAL_DATA_ COMPLEXITY (gdv)	Complexity derived from the use of global data	[R51], [R56]
Software Quality Metrics	GLOBAL_DATA_DENSITY (gd(G))	Density of access to global data relative to the total	[R51], [R56]
Software Quality Metrics	MAINTENANCE_SEVERITY	Severity in software maintenance	[R51], [R56]
Software Quality Metrics	HCM	Composite measure of complexity for maintenance	[R46]
Software Quality Metrics	WHCM	Weighted HCM	[R46]
Software Quality Metrics	LDHCM	Layered Depth of HCM	[R46]
Software Quality Metrics	LGDHCM	Generalized Depth of HCM	[R46]
Software Quality Metrics	EDHCM	Extended Depth of HCM	[R46]
Change History	NR	Number of revisions	[R46]
Change History	NFIX	Number of corrections made	[R46]
Change History	NREF	Number of references to previous errors	[R46]
Change History	NAUTH	Number of authors who modified the file	[R46]
Change History	LOC_ADDED	Lines of code added in a review	[R46]
Change History	maxLOC_ADDED	Maximum lines added in a single revision	[R46]
Change History	avgLOC_ADDED	Average lines added per review	[R46]
Change History	LOC_REMOVED	Total lines removed	[R46]
Change History	max LOC_REMOVED	Maximum number of lines removed in a revision	[R46]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Change History	avg LOC_REMOVED	Average number of lines removed per review	[R46]
Change History	AGE	Age of the file since its creation	[R46]
Change History	WAGE	Weighted age by the size of the modifications	[R46]
Change History	CVSEntropy	Entropy of repository change history	[R01], [R44]
Change History	numberOfNontrivialBugs FoundUntil	Cumulative number of significant bugs found	[R01]
Change History	Entropía mejorada	Refined variant of modification entropy	[R22]
Change History	fault	Total count of recorded failures	[R16], [R44]
Change History	Defects	Total number of defects recorded	[R15], [R46], [R10]
Defect History	Bugs	Count of bugs found or related to the file	[R46]
Change Metric	codeCHU	Code Change History Unit	[R46]
Change Metric	maxCodeCHU	Maximum codeCHU value in a review	[R46]
Change Metric	avgCodeCHU	Average codeCHU over time	[R46]
Descriptive statistics	mea	Average value (arithmetic mean)	[R22]
Descriptive statistics	median	Central value of the data distribution	[R22]
Descriptive statistics	SD	Standard deviation: dispersion of the data	[R22]
Descriptive statistics	Curtosis	Measure of the concentration of values in the mean	[R22]
Descriptive statistics	moments	Statistical moments of a distribution	[R22]
Descriptive statistics	skewness	Asymmetry of distribution	[R22]
MPI communication	send_num	Number of MPI submissions (blocking)	[R24]
MPI communication	recv_num	Number of MPI receptions	[R24]
MPI communication	Isend_num	Non-blocking MPI submissions	[R24]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
MPI communication	Irecv_num	Non-blocking MPI receptions	[R24]
MPI communication	recv_precedes_send	Reception occurs before dispatch	[R24]
MPI communication	mismatching_type, size	Incompatible types or sizes in communication	[R24]
MPI communication	any_source, any_tag	Using wildcards in MPI communication (MPI_ANY_SOURCE, etc.)	[R24]
MPI communication	recv_without_wait	Reception without active waiting (non-blocking)	[R24]
MPI communication	send_without_wait	Shipping without active waiting	[R24]
MPI communication	request_overwrite	Possible overwriting of MPI requests	[R24]
MPI communication	collective_order_issue	Order problems in collective operations	[R24]
MPI communication	collective_missing	Lack of required collective calls	[R24]
Syntactic Metrics	LCSAt	Total size of the Abstract Syntax Tree (AST)	[R29]
Syntactic Metrics	LCSAr	AST depth	[R29]
Syntactic Metrics	LCSAu	Number of unique nodes in the AST	[R29]
Syntactic Metrics	LCSAm	Average number of nodes per AST branch	[R29]
Syntactic Metrics	N_AST	Total number of nodes in the abstract syntax tree (AST)	[R41]
Textual semantics	Line + data/control flow	Logical representation of control/data flow	[R03]
Textual semantics	Doc2Vec vector (100 dimensions)	Vectorized textual embedding of source code	[R03]
Textual semantics	Token Vector	Tokenized representation of the code	[R24], [R63]
Textual semantics	Bag of Words	Word frequency-based representation	[R24]
Textual semantics	Padded Vector	Normalized vector with padding for neural networks	[R24]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Network Metrics	degree_norm, Katz_norm	Centrality metrics in dependency graphs	[R03]
Network Metrics	closeness_norm	Normalized closeness metric in dependency graph	[R03]
Concurrency Metric	reading_writing_same_buffer	Concurrent access to the same buffer	[R24]
Execution Dynamics	Relative execution time	Relationship between test duration and total sum	[R04], [R02]
Execution Dynamics	Execution history	binary vector with previous results: 0 = failed, 1 = passed	[R04]
Execution Dynamics	Last execution	normalized temporal proximity	[R04]
Interface Elements	EIem_Inter	Extracted interface elements	[R60], [R35], [R39]
Programs	Programs (Source code, test case sets, injected fault points, and running scripts.)	Program content	[R64]
Graphical models/state diagrams	State Transition Diagrams	OO Systems: Braille translator, microwave, and ATM	[R14]
Textual semantics	BoW	Represents the text by word frequency.	[R48]
Textual semantics	TF-IDF	Highlights words that are frequent in a text but rare in the corpus.	[R48]
Traces and calls	Function names	Names of the functions called in the trace	[R32]
Traces and calls	Return values	Return values of functions	[R32]
Traces and calls	Arguments	Input arguments used in each call	[R32]
Visuals/images	UI_images	Screenshots (UI) represented by images.	[R43]
Traces and calls	class name	Extracted and separated from JUnit classes in Java	[R62]
Traces and calls	Method name	Generated from test methods (@Test)	[R62]
Traces and calls	Method body	Tokenized source code	[R62]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Static code metrics	60 static metrics (calculated with OSA), originally 22 in some datasets.	Source code variables such as lines of code, cyclomatic complexity, and object-oriented metrics, used to predict defects.	[R42], [R06]
BDD Scenario/Text	BDD Scenario (Given-When text)	CSV generated from user stories	[R23], [R02]
GUI Visuals/Interface Processing	GUI images	Visuals (image) + derived structures (masks)	[R26]
Textual semantics	If conditions + tokens	Conditional fragments and tokenized structures for error handling classification.	[R63]
Embedded representation	Word2Vec embedding	Vector representation of source code for input to the classifier.	[R63]
Supervised labeling	Error-handling tag	Binary variable to train the classifier (error handling/normal)	[R63]
Embedded representation	Neural activations	Internal outputs of neurons in different layers of the model under test inputs	[R27]
Embedded representation	Active combinations	Sets of neurons activated simultaneously during execution	[R27]
Embedded representation	Hash combinations	Hash representation of active joins to speed up coverage evaluation (HashC-NC)	[R27
GUI interaction	Events (interaction sequences)	Clicks, keys pressed, sequence of actions	[R20]
Test set	Test Paths	Sets of events executed by a test case	[R20]
Textual semantics	Input sequence	Character sequence (fuzz inputs) processed by Bi-LSTM	[R37]
Fuzzing	Unique paths executed	Measure of structural effectiveness of the coverage test	[R37]
Fuzzing (search-based)	Entry Fitness	Probabilistic evaluation of the input value within GA	[R37]

Table A3. *Cont.*

Subcategory	Variable	Description	Study ID
Visuals/images	Activations of conv3_2 and conv4_2 layers	Vector representations of images extracted from VGGNet layers to measure diversity in fuzzing.	[R52]
Latent representations (autoencoding)	Autoencoder outputs, mutated inputs, latent distances	Mutated autoencoder representations evaluated for their effect on clustering.	[R25]
Integration Structure/OO Dependencies	Dependencies between classes, number of stubs generated, graph size	Relationships between classes and number of stubs needed to execute the proposed integration order.	[R61]
Mutant execution metrics	Number of test cases that kill the mutant, killability severity, mutated code, operator class	Statistical and structural attributes of mutants used as features to classify their ability to reveal real faults.	[R08]
Multisource (history + code)	04 features (52 code metrics, 8 clone metrics, 42 coding rule violations, 2 Git metrics)	Source code attributes and change history used to estimate fault proneness using MLP.	[R31]
Time sequence (interaction)	Sequence of player states (actions, objects, score, time, events)	Temporal game interaction variables used as input to an LSTM network to generate test events and evaluate gameplay.	[R54]
Structural combinatorics	Array size, levels per factor, coverage, mixed cardinalities	Combinatorial design parameters (values per factor and interaction strength) used to construct optimal test arrays via tabu search.	[R59]

Appendix D

Metrics used in AI studies for ST.

Table A4. Description of classic variables.

Discipline	Description	Metrics/Formula	Study ID
Classic performance	Proportion of correct predictions out of the total number of cases evaluated.	$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$	[R22], [R24], [R11], [R15], [R44], [R51], [R53], [R55], [R57], [R07], [R09], [R17], [R21], [R38], [R40], [R49], [R34], [R43], [R63], [R37], [R08], [R42], [R02], [R10], [R19], [R06]

Table A4. *Cont.*

Discipline	Description	Metrics/Formula	Study ID
Classic performance	Measures the proportion of true positives among all positive predictions made.	$Precision = \frac{TP}{TP+FP}$	[R22], [R24], [R11], [R15], [R16], [R42], [R28], [R29], [R55], [R57], [R65], [R07], [R09], [R21], [R49], [R66], [R60], [R32], [R63], [R08], [R02], [R13], [R10], [R19], [R06]
Classic performance	Evaluates the model's ability to correctly identify all positive cases.	$Recall = Sensitivity = TPR = \frac{TP}{TP+FN}$	[R22], [R24], [R11], [R15], [R42], [R18], [R29], [R50], [R55], [R57], [R65], [R07], [R09], [R21], [R37], [R40], [R49], [R66], [R60], [R32], [R63], [R08], [R02], [R10], [R19], [R06]
Classic performance	Harmonious balance between precision and recall, useful in scenarios with unbalanced classes.	$\mathrm{F}_{1-\mathrm{Score}} = 2 \times \frac{\mathrm{Precision} \times \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$	[R22], [R11], [R15], [R16], [R42], [R28], [R47], [R29], [R41], [R44], [R51], [R53], [R55], [R65], [R07], [R40], [R49], [R66], [R60], [R63], [R08], [R02], [R10], [R19], [R06]
Advanced Classification	Evaluates the quality of predictions considering true and false positives and negatives.	$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{11}}$	[R03], [R22], [R28], [R51], [R53], [R65], [R33], [R66]
Advanced Classification	Summarizes the model's ability to discriminate between positive and negative classes at different thresholds	$\mathrm{AUC} = \int_0^1 \mathrm{TPR}(\mathrm{FPR})\, \mathrm{d}(\mathrm{FPR})$	[R01], [R03], [R16], [R42], [R18], [R28], [R29], [R30], [R41], [R44], [R51], [R55], [R57], [R65], [R07], [R38], [R40], [R48], [R08], [R19], [R06]
Advanced Classification	Averages sensitivity and specificity, useful when classes are unbalanced.	$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right)$	[R03]
Advanced Classification	Geometric between sensitivity and specificity, measures the balance in binary classification.	$G\text{−Mean} = \sqrt{\frac{TP}{TP+FN} \times \frac{TN}{TN+FP}}$	[R03], [R16], [R18], [R55], [R65], [R33], [R46]
Alarms and Risk	Measures the proportion of true negatives detected among all true negative cases.	$Specificity = TNR = \frac{TN}{TN+FP}$	[R22], [R15], [R55], [R57], [R09], [R21], [R32], [R40]
Alarms and Risk	Proportion of true negatives among all negative predictions.	$NPV = \frac{TN}{TN+FN}$	[R22], [R09], [R21]
Alarms and Risk	Proportion of false positives among all positive predictions.	$\mathrm{FDR} = \frac{FP}{FP+TP}$	[R22]
Alarms and Risk	Proportion of undetected positives among all true positives.	$FNR = \frac{FN}{FN+TP}$	[R22], [R12], [R57], [R09], [R21], [R33]
Alarms and Risk	Proportion of negatives incorrectly classified as positives.	$\mathrm{FPR} = \frac{FP}{FP+TN}$	[R18], [R22], [R12], [R18], [R50], [R57], [R65], [R09], [R21], [R33]. [R18], [R37]
Software Testing-Specific Metrics	Measures the effort required (in percentage of LOC or files) to reach 20% recall.	$E@20R = \frac{\mathrm{LOC}_{20R}}{\mathrm{LOC}_{tot}}$	[R03]
Software Testing-Specific Metrics	Percentage of defects found within the 20% most suspicious lines of code.	$R@20E = \frac{D_{20E}}{D_{tot}}$	[R03]

Table A4. *Cont.*

Discipline	Description	Metrics/Formula	Study ID
Software Testing-Specific Metrics	Number of false positives before finding the first true positive.	$IFA = N_{\text{non-def}}^{\text{before 1st defect}}$	[R03], [R06]
Software Testing-Specific Metrics	Accuracy among the k elements best ranked by the model.	$A@k = \frac{P_k^{corr}}{N_{tot}}$	[R03]
Software Testing-Specific Metrics	Effort metric that combines precision and recall with weighting of the inspected code.	$P_{opt} = 1 - \frac{\text{Area}(M_{opt}) - \text{Area}(M_{model})}{\text{Area}(M_{opt}) - \text{Area}(M_{worst})}$	[R44]
Software Testing-Specific Metrics	It is used to compare how effectively a model detects faults early relative to a baseline model.	$\text{Norm} \mid \text{P}_{\text{opt}} = \frac{\text{P}_{\text{opt}}^{\text{model}} - \text{P}_{\text{opt}}^{\text{baseline}}}{1 - \text{P}_{\text{opt}}^{\text{baseline}}}$	[R04]
Software Testing-Specific Metrics	Expected number of test cases generated until the first failure is detected.	$E\left[T_f\right] = \sum_{i=1}^{n} i \times P\left(T_f = i\right)$	[R52]
Software Testing-Specific Metrics	Number of rows needed to cover all combinations t	$\text{MCA}(\text{N}; \text{t}, \text{k}, (\text{v}_1, \text{v}_2, \ldots, \text{v}_\text{k}))$	[R59]
Software Testing-Specific Metrics	Time required by MiTS to build the array	$\text{Coverage} - \text{way} = \frac{\text{Rows generated}}{\text{All possible combinations}} \times 100\backslash\%$	[R59]
Software Testing-Specific Metrics	Improvement compared to the best previously known values	$\Delta\text{Improvement} = \frac{\text{V}_{\text{new}} - \text{V}_{\text{best}}}{\text{V}_{\text{best}}} \times 100$	[R59]
Cost/Error and Probabilistic Metrics	Measures the mean square error between predicted probabilities and actual outcomes (lower is better).	$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_i - y_i)^2$	[R16]
Cost/Error and Probabilistic Metrics	Distance of the model to an ideal classifier with 100% TPR and 0% FPR.	$D_{\text{ROC}} = \sqrt{(1 - \text{TPR})^2 + (\text{FPR})^2}$	[R16]
Cost/Error and Probabilistic Metrics	Root mean square error between predicted and actual values; useful for regression models.	$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (\hat{y}_l - y_i)^2}$	[R53]
Cost/Error and Probabilistic Metrics	Expected time it takes for the model to detect a positive instance (defect) correctly.	$ETT = \sum_{i=1}^{n} e_i \cdot P_i$	[R53]
Cost/Error and Probabilistic Metrics	Ratio between the actual effort needed to achieve a certain recall and the optimal possible effort.	$OE = \frac{\text{Area under the stress-defect curve}}{\text{Area under the ideal stress-defect curve}}$	[R57]
Cost/Error and Probabilistic Metrics	Proportion of incorrectly classified instances relative to the total.	$\text{MR} = \frac{\text{FP+FN}}{\text{TP+TN+FP+FN}}$	[R09], [R21], [R56]
Coverage, Execution, GUI, and Deep Learning	Evaluates the speed of test point coverage. The closer to 1, the better.	$APTC = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{F_i}{T_i}\right)$	[R64]
Coverage, Execution, GUI, and Deep Learning	Evaluate the total runtime until full coverage is achieved. The lower the better.	$EET = \sum_{i=1}^{m} t_i$	[R64]

Table A4. *Cont.*

Discipline	Description	Metrics/Formula	Study ID
Coverage, Execution, GUI, and Deep Learning	Evaluates the similarity between a generated text (e.g., test case) and a reference text, using n-gram matches and brevity penalties.	$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$	[R35], [R39], [R62]
Coverage, Execution, GUI, and Deep Learning	Measures the average accuracy of the model in object detection at different matching thresholds (IoU).	$\text{mAP} = \frac{1}{N}\sum_{i=1}^{N} \text{AP}_i$	[R39]
Coverage, Execution, GUI, and Deep Learning	Measures the total time it takes for an algorithm to generate all test paths.	$T_{\text{gen}} = t_{\text{end}} - t_{\text{start}}$	[R14], [R20], [R25], [R27], [R37], [R61]
Coverage, Execution, GUI, and Deep Learning	Indicates the proportion of repeated or unnecessary test paths generated by the algorithm.	$\text{Redundancy} = \frac{N_{\text{redundant}}}{N_{\text{generated}}}$	[R14]
Coverage, Execution, GUI, and Deep Learning	Fraction of generated step methods that have implementation	$ImplementedFraction = \frac{N_{\text{implemented steps}}}{N_{\text{total generated steps}}}$	[R23]
Coverage, Execution, GUI, and Deep Learning	Fraction of generated step methods without implementation	$NonImplementedFraction = \frac{N_{\text{non-implemented steps}}}{N_{\text{total generated steps}}}$	[R23]
Coverage, Execution, GUI, and Deep Learning	Fraction of generated POM methods with functional implementation	$FunctionalFraction = \frac{N_{\text{functional POM methods}}}{N_{\text{total POM methods}}}$	[R23]
Coverage, Execution, GUI, and Deep Learning	Average number of paths covered by the algorithm	$\text{AC} = \frac{1}{n}\sum_{i=1}^{n} \text{Coverage}_i$	[R36], [R05]
Coverage, Execution, GUI, and Deep Learning	Average number of generations needed to cover all paths	$\text{AG} = \frac{1}{n}\sum_{i=1}^{n} \text{Generations}_i$	[R36], [R05]
Coverage, Execution, GUI, and Deep Learning	Percentage of executions that cover all paths	$\text{Coverage}\backslash\% = SR = \frac{N_{\text{successful executions}}}{N_{\text{total executions}}} \times 100$	[R36], [R05]
Coverage, Execution, GUI, and Deep Learning	Average execution time of the algorithm	$\text{AT} = \frac{1}{n}\sum_{i=1}^{n} \text{ExecutionTime}_i$	[R36], [R05]
Coverage, Execution, GUI, and Deep Learning	It is equivalent to an accuracy metric, applied to a visual matching task.	$Correct\ Rate = \frac{N_{\text{correct predictions}}}{N_{\text{total predictions}}}$	[R26]
Coverage, Execution, GUI, and Deep Learning	Measures how many unique neural combinations have been covered	$HashC\ Coverage = \frac{N_{\text{covered buckets}}}{N_{\text{total buckets}}}$	[R27]
Coverage, Execution, GUI, and Deep Learning	Measures whether a neuron was activated at least once	$\text{NC} = \frac{N_{\text{activated neurons}}}{N_{\text{total neurons}}} \times 100$	[R27]
Coverage, Execution, GUI, and Deep Learning	Coverage of combinations of 2 neurons activated together	$\text{2NC} = \frac{N_{\text{2-neuron activations}}}{N_{\text{total 2-neuron combinations}}} \times 100$	[R27]
Coverage, Execution, GUI, and Deep Learning	Coverage of combinations of 3 neurons activated together	$\text{3NC} = \frac{N_{\text{3-neuron activations}}}{N_{\text{total 3-neuron combinations}}} \times 100$	[R27]
Coverage, Execution, GUI, and Deep Learning	Percentage of test paths covered by the generated test cases	$PathCoverage = \frac{N_{\text{covered test paths}}}{N_{\text{total test paths}}} \times 100$	[R20]
Coverage, Execution, GUI, and Deep Learning	% of unique events covered (equivalent to coverage by GUI widgets)	$\text{Coverage} = \frac{N_{\text{covered events}}}{N_{\text{total events}}} \times 100$	[R20]

Table A4. *Cont.*

Discipline	Description	Metrics/Formula	Study ID
Coverage, Execution, GUI, and Deep Learning	Percentage of code executed during testing.	$CodeCoverage = \frac{N_{\text{lines executed}}}{N_{\text{total lines}}} \times 100$	[R37]
Coverage, Execution, GUI, and Deep Learning	Weighted measure of coverage diversity among generated cases.	$\text{WDC} = \sum_{i=1}^{n} w_i \times \text{coverage}_i$	[R37]
Coverage, Execution, GUI, and Deep Learning	Proportion of mutants detected per change in system output	$MutationScore = \frac{N_{\text{mutants detected}}}{N_{\text{total mutants}}}$	[R25]
Coverage, Execution, GUI, and Deep Learning	Euclidean distance in latent space between original and mutated input	$L_2 = \sqrt{\sum_{i=1}^{n} (x_i - x_i')^2}$	[R25]
Coverage, Execution, GUI, and Deep Learning	Total number of stubs needed for each order	$TotalStubs = \sum_{i=1}^{n} S_i$	[R61]
Coverage, Execution, GUI, and Deep Learning	Reduction in the number of stubs compared to baseline	$SavingRate = \frac{S_{\text{baseline}} - S_{\text{proposed}}}{S_{\text{baseline}}} \times 100$	[R61]
Coverage, Execution, GUI, and Deep Learning	Evaluate the effectiveness of test case prioritization	$\text{APFD} = 1 - \frac{\sum_{i=1}^{m} T_i}{n \times m} + \frac{1}{2n}$	[R31]
Coverage, Execution, GUI, and Deep Learning	Percentage of LSTM predictions that match expected gameplay	$CorrectPredictions = \frac{N_{\text{correct predictions}}}{N_{\text{total predictions}}} \times 100$	[R54]
Coverage, Execution, GUI, and Deep Learning	Measure of balance between the actions and responses of the game	$BalanceScore = \frac{1}{N} \sum_{i=1}^{N} S_i$	[R54]

References

1. Manyika, J.; Chui, M.; Bughin, J.; Dobbs, R.; Bisson, P.; Marrs, A. *Disruptive Technologies: Advances That Will Transform Life, Business, and the Global Economy*; McKinsey Global Institute: San Francisco, CA, USA, 2013. Available online: https://www.mckinsey.com/mgi/overview (accessed on 3 November 2025).
2. Hameed, K.; Naha, R.; Hameed, F. Digital transformation for sustainable health and well-being: A review and future research directions. *Discov. Sustain.* **2024**, *5*, 104. [CrossRef]
3. Software & Information Industry Association (SIIA). The Software Industry: Driving Growth and Employment in the U.S. Economy. 2020. Available online: https://www.siia.net/ (accessed on 31 October 2025).
4. Anderson, R. *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed.; John Wiley & Sons: Hoboken, NJ, USA, 2020. [CrossRef]
5. Clark, R.C.; Mayer, R.E. *E-Learning and the Science of Instruction: Proven Guidelines for Consumers and Designers of Multimedia Learning*, 4th ed.; John Wiley & Sons: Hoboken, NJ, USA, 2016.
6. Saxena, A. Rethinking Software Testing for Modern Development. *Computer* **2025**, *58*, 49–58. [CrossRef]
7. Karvonen, J. Enhancing Software Quality: A Comprehensive Study of Modern Software Testing Methods. Unpublished Doctoral Dissertation. Ph.D. Thesis, Tampere University, Tampere, Finland, 2024.
8. Kazimov, T.H.; Bayramova, T.A.; Malikova, N.J. Research of intelligent methods of software testing. *Syst. Res. Inf. Technol.* **2022**, 42–52. [CrossRef]
9. Arunachalam, M.; Kumar Babu, N.; Perumal, A.; Ohnu Ganeshbabu, R.; Ganesh, J. Cross-layer design for combining adaptive modulation and coding with DMMPP queuing for wireless networks. *J. Comput. Sci.* **2023**, *19*, 786–795. [CrossRef]
10. Gao, J.; Tsao, H.; Wu, Y. *Testing and Quality Assurance for Component-Based Software*; Artech House: Norwood, MA, USA, 2006.
11. Lima, B. Automated Scenario-Based Integration Testing of Time-Constrained Distributed Systems. In Proceedings of the 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST), Xi'an, China, 22–27 April 2019; pp. 486–488. [CrossRef]
12. Fontes, A.; Gay, G. The integration of machine learning into automated test generation: A systematic mapping study. *arXiv* **2023**, arXiv:2206.10210. [CrossRef]
13. Sharma, C.; Sabharwal, S.; Sibal, R. A survey on software testing techniques using genetic algorithm. *arXiv* **2014**, arXiv:1411.1154. [CrossRef]

14. Juneja, S.; Taneja, H.; Patel, A.; Jadhav, Y.; Saroj, A. Bio-inspired optimization algorithm in machine learning and practical applications. *SN Comput. Sci.* **2024**, *5*, 1081. [CrossRef]
15. Menzies, T.; Greenwald, J.; Frank, A. Data mining static code attributes to learn defect predictors. *IEEE Trans. Softw. Eng.* **2007**, *33*, 2–13. [CrossRef]
16. Zimmermann, T.; Premraj, R.; Zeller, A. Cross-project defect prediction: A large-scale experiment on open-source projects. In Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering, Amsterdam, The Netherlands, 24–28 August 2009; pp. 91–100. [CrossRef]
17. Khaliq, Z.; Farooq, S.U.; Khan, D.A. A deep learning-based automated framework for functional User Interface testing. *Inf. Softw. Technol.* **2022**, *150*, 106969. [CrossRef]
18. Sreedevi, E.; Kavitha, P.; Mani, K. Performance of heterogeneous ensemble approach with traditional methods based on software defect detection model. *J. Theor. Appl. Inf. Technol.* **2022**, *100*, 980–989.
19. Khaliq, Z.; Farooq, S.U.; Khan, D.A. Using deep learning for selenium web UI functional tests: A case-study with e-commerce applications. *Eng. Appl. Artif. Intell.* **2023**, *117*, 105446. [CrossRef]
20. Borandag, E. Software fault prediction using an RNN-based deep learning approach and ensemble machine learning techniques. *Appl. Sci.* **2023**, *13*, 1639. [CrossRef]
21. Stradowski, S.; Madeyski, L. Machine learning in software defect prediction: A business-driven systematic mapping study. *Inf. Softw. Technol.* **2023**, *155*, 107128. [CrossRef]
22. Amalfitano, D.; Faralli, S.; Rossa Hauck, J.C.; Matalonga, S.; Distante, D. Artificial intelligence applied to software testing: A tertiary study. *ACM Comput. Surv.* **2024**, *56*, 1–38. [CrossRef]
23. Boukhlif, M.; Hanine, M.; Kharmoum, N.; Ruigómez Noriega, A.; García Obeso, D.; Ashraf, I. Natural language processing-based software testing: A systematic literature review. *IEEE Access* **2024**, *12*, 79383–79400. [CrossRef]
24. Ajorloo, S.; Jamarani, A.; Kashfi, M.; Haghi Kashani, M.; Najafizadeh, A. A systematic review of machine learning methods in software testing. *Appl. Soft Comput.* **2024**, *162*, 111805. [CrossRef]
25. Salahirad, A.; Gay, G.; Mohammadi, E. Mapping the structure and evolution of software testing research over the past three decades. *J. Syst. Softw.* **2023**, *195*, 111518. [CrossRef]
26. Peischl, B.; Tazl, O.A.; Wotawa, F. Testing anticipatory systems: A systematic mapping study on the state of the art. *J. Syst. Softw.* **2022**, *192*, 111387. [CrossRef]
27. Khokhar, M.N.; Bashir, M.B.; Fiaz, M. Metamorphic testing of AI-based applications: A critical review. *Int. J. Adv. Comput. Sci. Appl.* **2020**, *11*, 754–761. [CrossRef]
28. Khatibsyarbini, M.; Isa, M.A.; Jawawi, D.N.A.; Shafie, M.L.M.; Wan-Kadir, W.M.N. Trend application of machine learning in test case prioritization: A review on techniques. *IEEE Access* **2021**, *9*, 166262–166282. [CrossRef]
29. Boukhlif, M.; Hanine, M.; Kharmoum, N. A decade of intelligent software testing research: A bibliometric analysis. *Electronics* **2023**, *12*, 2109. [CrossRef]
30. Myers, G.J. *The Art of Software Testing*; Wiley-Interscience: New York, NY, USA, 1979.
31. *ISO/IEC/IEEE 29119-1:2013*; Software and Systems Engineering—Software Testing—Part 1: Concepts and Definitions. International Organization for Standardization: Geneva, Switzerland, 2013.
32. Kaner, C.; Bach, J.; Pettichord, B. *Testing Computer Software*, 2nd ed.; John Wiley & Sons: New York, NY, USA, 2002.
33. Pressman, R.S.; Maxim, B.R. *Software Engineering: A Practitioner's Approach*, 8th ed.; McGraw-Hill Education: New York, NY, USA, 2014.
34. Boehm, B.; Basili, V.R. Top 10 list [software development]. *Computer* **2001**, *34*, 135–137. [CrossRef]
35. McGraw, G. *Software Security: Building Security*; Addison-Wesley Professional: Boston, MA, USA, 2006.
36. Beizer, B. *Software Testing Techniques*, 2nd ed.; Van Nostrand Reinhold: New York, NY, USA, 1990.
37. Kan, S.H. *Metrics and Models in Software Quality Engineering*, 2nd ed.; Addison-Wesley: Boston, MA, USA, 2002.
38. Beck, K. *Test Driven Development: By Example*; Addison-Wesley: Boston, MA, USA; Longman: Harlow, UK, 2002.
39. Humble, J.; Farley, D. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*; Addison-Wesley Professional: Boston, MA, USA, 2010.
40. Jorgensen, P.C. *Software Testing: A Craftsman's Approach*, 4th ed.; CRC Press: Boca Raton, FL, USA, 2013.
41. Crispin, L.; Gregory, J. *Agile Testing: A Practical Guide for Testers and Agile Teams*; Addison-Wesley: Boston, MA, USA, 2009; Available online: https://books.google.com/books?id=3UdsAQAAQBAJ (accessed on 31 October 2025).
42. Graham, D.; Fewster, M. *Experiences of Test Automation: Case Studies of Software Test Automation*; Addison-Wesley: Boston, MA, USA, 2012.
43. Meier, J.D.; Farre, C.; Bansode, P.; Barber, S.; Rea, D. Performance Testing Guidance for Web Applications, 1st ed.Microsoft Press: Redmond, WA, USA, 2007.
44. North, D. Introducing BDD. 2006. Available online: https://dannorth.net/introducing-bdd/ (accessed on 31 October 2025).
45. Fewster, M.; Graham, D. *Software Test Automation*; Addison-Wesley: Boston, MA, USA, 1999.

46. Pelivani, E.; Cico, B. A comparative study of automation testing tools for web applications. In Proceedings of the 2021 10th Mediterranean Conference on Embedded Computing (MECO), Budva, Montenegro, 7–11 June 2021; pp. 1–6. [CrossRef]
47. Beck, K.; Saff, D. *JUnit Pocket Guide*; O'Reilly Media: Sebastopol, CA, USA, 2004.
48. Black, R. Advanced Software Testing. In *Guide to the ISTQB Advanced Certification as an Advanced Test Analyst*, 2nd ed.; Rocky Nook: Santa Barbara, CA, USA, 2009; Volume 1.
49. Kitchenham, B. *Software Metrics: Measurement for Software Process Improvement*; John Wiley & Sons: Chichester, UK, 1996.
50. Cohn, M. *Agile Estimating and Planning*; Pearson Education: Upper Saddle River, NJ, USA, 2005.
51. Harman, M.; Mansouri, S.A.; Zhang, Y. Search-based software engineering: Trends, techniques and applications. *ACM Comput. Surv.* **2012**, *45*, 11. [CrossRef]
52. Arora, L.; Girija, S.S.; Kapoor, S.; Raj, A.; Pradhan, D.; Shetgaonkar, A. Explainable artificial intelligence techniques for software development lifecycle: A phase-specific survey. In Proceedings of the 2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC), Toronto, ON, Canada, 8–11 July 2025; pp. 2281–2288. [CrossRef]
53. Kitchenham, B.; Charters, S. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*; EBSE Technical Report, Ver. 2.3; Keele University: Staffordshire, UK; University of Durham: Durham, UK, 2007.
54. Marinescu, R.; Seceleanu, C.; Guen, H.L.; Pettersson, P. Chapter Three—A Research Overview of Tool-Supported Model-Based Testing of Requirements-Based Designs. In *Advances in Computers*; Hurson, A.R., Ed.; Elsevier: Amsterdam, The Netherlands, 2015; Volume 98, pp. 89–140. [CrossRef]
55. Garousi, V.; Mäntylä, M.V. A systematic literature review of literature reviews in software testing. *Inf. Softw. Technol.* **2016**, *80*, 195–216. [CrossRef]
56. Arcos-Medina, G.; Mauricio, D. Aspects of software quality applied to the process of agile software development: A systematic literature review. *Int. J. Syst. Assur. Eng. Manag.* **2019**, *10*, 867–897. [CrossRef]
57. Pachouly, J.; Ahirrao, S.; Kotecha, K.; Selvachandran, G.; Abraham, A. A systematic literature review on software defect prediction using artificial intelligence: Datasets, data validation methods, approaches, and tools. *Eng. Appl. Artif. Intell.* **2022**, *111*, 104773. [CrossRef]
58. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. [CrossRef]
59. Malhotra, R.; Khan, K. A novel software defect prediction model using two-phase grey wolf optimization for feature selection. *Clust. Comput.* **2024**, *27*, 12185–12207. [CrossRef]
60. Zulkifli, Z.; Gaol, F.L.; Trisetyarso, A.; Budiharto, W. Software Testing Integration-Based Model (I-BM) framework for recognizing measure fault output accuracy using machine learning approach. *Int. J. Softw. Eng. Knowl. Eng.* **2023**, *33*, 1149–1168. [CrossRef]
61. Yang, F.; Zhong, F.; Zeng, G.; Xiao, P.; Zheng, W. LineFlowDP: A deep learning-based two-phase approach for line-level defect prediction. *Empir. Softw. Eng.* **2024**, *29*, 50. [CrossRef]
62. Rosenbauer, L.; Pätzel, D.; Stein, A.; Hähner, J. A learning classifier system for automated test case prioritization and selection. *SN Comput. Sci.* **2022**, *3*, 373. [CrossRef]
63. Ghaemi, A.; Arasteh, B. SFLA-based heuristic method to generate software structural test data. *J. Softw. Evolu. Process* **2020**, *32*, e2228. [CrossRef]
64. Zhang, S.; Jiang, S.; Yan, Y. A hierarchical feature ensemble deep learning approach for software defect prediction. *Int. J. Softw. Eng. Knowl. Eng.* **2023**, *33*, 543–573. [CrossRef]
65. Ali, M.; Mazhar, T.; Al-Rasheed, A.; Shahzad, T.; Ghadi, Y.Y.; Khan, M.A. Enhancing software defect prediction: A framework with improved feature selection and ensemble machine learning. *PeerJ Comput. Sci.* **2024**, *10*, e1860. [CrossRef]
66. Rostami, T.; Jalili, S. FrMi: Fault-revealing mutant identification using killability severity. *Inf. Softw. Technol.* **2023**, *164*, 107307. [CrossRef]
67. Ali, M.; Mazhar, T.; Arif, Y.; Al-Otaibi, S.; Yasin Ghadi, Y.; Shahzad, T.; Khan, M.A.; Hamam, H. Software defect prediction using an intelligent ensemble-based model. *IEEE Access* **2024**, *12*, 20376–20395. [CrossRef]
68. Gangwar, A.K.; Kumar, S. Concept drift in software defect prediction: A method for detecting and handling the drift. *ACM Trans. Internet Technol.* **2023**, *23*, 1–28. [CrossRef]
69. Wang, H.; Arasteh, B.; Arasteh, K.; Gharehchopogh, F.S.; Rouhi, A. A software defect prediction method using binary gray wolf optimizer and machine learning algorithms. *Comput. Electr. Eng.* **2024**, *118*, 109336. [CrossRef]
70. Abaei, G.; Selamat, A. Increasing the accuracy of software fault prediction using majority ranking fuzzy clustering. In *Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing*; Lee, R., Ed.; Springer International Publishing: Cham, Switzerland, 2015; pp. 179–193. [CrossRef]
71. Qiu, S.; Huang, H.; Jiang, W.; Zhang, F.; Zhou, W. Defect prediction via tree-based encoding with hybrid granularity for software sustainability. *IEEE Trans. Sustain. Comput.* **2024**, *9*, 249–260. [CrossRef]

72. Sharma, R.; Saha, A. Optimal test sequence generation in state based testing using moth flame optimization algorithm. *J. Intell. Fuzzy Syst.* **2018**, *35*, 5203–5215. [CrossRef]
73. Jayanthi, R.; Florence, M.L. Improved Bayesian regularization using neural networks based on feature selection for software defect prediction. *Int. J. Comput. Appl. Technol.* **2019**, *60*, 216–224. [CrossRef]
74. Nikravesh, N.; Keyvanpour, M.R. Parameter tuning for software fault prediction with different variants of differential evolution. *Expert Syst. Appl.* **2024**, *237*, 121251. [CrossRef]
75. Mehmood, I.; Shahid, S.; Hussain, H.; Khan, I.; Ahmad, S.; Rahman, S.; Ullah, N.; Huda, S. A novel approach to improve software defect prediction accuracy using machine learning. *IEEE Access* **2023**, *11*, 63579–63597. [CrossRef]
76. Chen, L.; Fang, B.; Shang, Z.; Tang, Y. Tackling class overlap and imbalance problems in software defect prediction. *Softw. Qual. J.* **2018**, *26*, 97–125. [CrossRef]
77. Rajnish, K.; Bhattacharjee, V. A cognitive and neural network approach for software defect prediction. *J. Intell. Fuzzy Syst.* **2022**, *43*, 6477–6503. [CrossRef]
78. Abbas, S.; Aftab, S.; Khan, M.A.; Ghazal, T.M.; Hamadi, H.A.; Yeun, C.Y. Data and ensemble machine learning fusion based intelligent software defect prediction system. *Comput. Mater. Contin.* **2023**, *75*, 6083–6100. [CrossRef]
79. Al-Johany, N.A.; Eassa, F.; Sharaf, S.A.; Noaman, A.Y.; Ahmed, A. Prediction and correction of software defects in Message-Passing Interfaces using a static analysis tool and machine learning. *IEEE Access* **2023**, *11*, 60668–60680. [CrossRef]
80. Lu, Y.; Shao, K.; Zhao, J.; Sun, W.; Sun, M. Mutation testing of unsupervised learning systems. *J. Syst. Archit.* **2024**, *146*, 103050. [CrossRef]
81. Zhang, L.; Tsai, W.-T. Adaptive attention fusion network for cross-device GUI element re-identification in crowdsourced testing. *Neurocomputing* **2024**, *580*, 127502. [CrossRef]
82. Sun, W.; Xue, X.; Lu, Y.; Zhao, J.; Sun, M. HashC: Making deep learning coverage testing finer and faster. *J. Syst. Archit.* **2023**, *144*, 102999. [CrossRef]
83. Pandey, S.K.; Singh, K.; Sharma, S.; Saha, S.; Suri, N.; Gupta, N. Software defect prediction using K-PCA and various kernel-based extreme learning machine: An empirical study. *IET Softw.* **2020**, *14*, 768–782. [CrossRef]
84. Li, Z.; Wang, X.; Zhang, Y.; Liu, T.; Chen, J. Software defect prediction based on hybrid swarm intelligence and deep learning. *Comput. Intell. Neurosci.* **2021**, *2021*, 4997459. [CrossRef] [PubMed]
85. Singh, P.; Verma, S. ACO based comprehensive model for software fault prediction. *Int. J. Knowl. Based Intell. Eng. Syst.* **2020**, *24*, 63–71. [CrossRef]
86. Manikkannan, D.; Babu, S. Automating software testing with multi-layer perceptron (MLP): Leveraging historical data for efficient test case generation and execution. *Int. J. Intell. Syst. Appl. Eng.* **2023**, *11*, 424–428.
87. Tsimpourlas, F.; Rooijackers, G.; Rajan, A.; Allamanis, M. Embedding and classifying test execution traces using neural networks. *IET Softw.* **2022**, *16*, 301–316. [CrossRef]
88. Kumar, G.; Chopra, V. Hybrid approach for automated test data generation. *J. ICT Stand.* **2022**, *10*, 531–562. [CrossRef]
89. Ma, M.; Han, L.; Qian, Y. CVDF DYNAMIC—A dynamic fuzzy testing sample generation framework based on BI-LSTM and genetic algorithm. *Sensors* **2022**, *22*, 1265. [CrossRef] [PubMed]
90. Sangeetha, M.; Malathi, S. Modeling metaheuristic optimization with deep learning software bug prediction model. *Intell. Autom. Soft Comput.* **2022**, *34*, 1587–1601. [CrossRef]
91. Zada, I.; Alshammari, A.; Mazhar, A.A.; Aldaeej, A.; Qasem, S.N.; Amjad, K.; Alkhateeb, J.H. Enhancing IoT-based software defect prediction in analytical data management using war strategy optimization and kernel ELM. *Wirel. Netw.* **2024**, *30*, 7207–7225. [CrossRef]
92. Šikić, L.; KurdiJA, A.S.; Vladimir, K.; Šilić, M. Graph neural network for source code defect prediction. *IEEE Access* **2022**, *10*, 10402–10415. [CrossRef]
93. Hai, T.; Chen, Y.; Chen, R.; Nguyen, T.N.; Vu, M. Cloud-based bug tracking software defects analysis using deep learning. *J. Cloud Comput.* **2022**, *11*, 32. [CrossRef]
94. Widodo, A.P.; Marji, A.; Ula, M.; Windarto, A.P.; Winarno, D.P. Enhancing software user interface testing through few-shot deep learning: A novel approach for automated accuracy and usability evaluation. *Int. J. Adv. Comput. Sci. Appl.* **2023**, *14*, 578–585. [CrossRef]
95. Fatima, S.; Hassan, S.; Zhang, H.; Dang, Y.; Nadi, S.; Hassan, A.E. Flakify: A black-box, language model-based predictor for flaky tests. *IEEE Trans. Softw. Eng.* **2023**, *49*, 1912–1927. [CrossRef]
96. Borandag, E.; Altınel, B.; Kutlu, B. Majority vote feature selection algorithm in software fault prediction. *Comput. Sci. Inf. Syst.* **2019**, *16*, 515–539. [CrossRef]
97. Mesquita, D.P.P.; Rocha, L.S.; Gomes, J.P.P.; Rocha Neto, A.R. Classification with reject option for software defect prediction. *Appl. Soft Comput.* **2016**, *49*, 1085–1093. [CrossRef]
98. Tahvili, S.; Garousi, V.; Felderer, M.; Pohl, J.; Heldal, R. A novel methodology to classify test cases using natural language processing and imbalanced learning. *Eng. Appl. Artif. Intell.* **2020**, *95*, 103878. [CrossRef]

99. Sharma, K.K.; Sinha, A.; Sharma, A. Software defect prediction using deep learning by correlation clustering of testing metrics. *Int. J. Electr. Comput. Eng. Syst.* **2022**, *13*, 953–960. [CrossRef]
100. Wójcicki, B.; Dąbrowski, R. Applying machine learning to software fault prediction. *e-Inform. Softw. Eng. J.* **2018**, *12*, 199–216. [CrossRef]
101. Matloob, F.; Aftab, S.; Iqbal, A. A framework for software defect prediction using feature selection and ensemble learning techniques. *Int. J. Mod. Educ. Comput. Sci. (IJMECS)* **2019**, *11*, 14–20. [CrossRef]
102. Yan, M.; Wang, L.; Fei, A. ARTDL: Adaptive random testing for deep learning systems. *IEEE Access* **2020**, *8*, 3055–3064. [CrossRef]
103. Yohannese, C.W.; Li, T.; Bashir, K. A three-stage based ensemble learning for improved software fault prediction: An empirical comparative study. *Int. J. Comput. Intell. Syst.* **2018**, *11*, 1229–1247. [CrossRef]
104. Chen, L.-K.; Chen, Y.-H.; Chang, S.-F.; Chang, S.-C. A Long/Short-Term Memory based automated testing model to quantitatively evaluate game design. *Appl. Sci.* **2020**, *10*, 6704. [CrossRef]
105. Ma, B.; Zhang, H.; Chen, G.; Zhao, Y.; Baesens, B. Investigating associative classification for software fault prediction: An experimental perspective. *Int. J. Softw. Eng. Knowl. Eng.* **2014**, *24*, 61–90. [CrossRef]
106. Singh, P.; Pal, N.R.; Verma, S.; Vyas, O.P. Fuzzy rule-based approach for software fault prediction. *IEEE Trans. Syst. Man Cybern. Syst.* **2017**, *47*, 826–837. [CrossRef]
107. Miholca, D.-L.; Czibula, G.; Czibula, I.G. A novel approach for software defect prediction through hybridizing gradual relational association rules with artificial neural networks. *Inf. Sci.* **2018**, *441*, 152–170. [CrossRef]
108. Guo, S.; Chen, R.; Li, H. Using knowledge transfer and rough set to predict the severity of Android test reports via text mining. *Symmetry* **2017**, *9*, 161. [CrossRef]
109. Gonzalez-Hernandez, L. New bounds for mixed covering arrays in t-way testing with uniform strength. *Inf. Softw. Technol.* **2015**, *59*, 17–32. [CrossRef]
110. Sharma, M.M.; Agrawal, A.; Kumar, B.S. Test case design and test case prioritization using machine learning. *Int. J. Eng. Adv. Technol.* **2019**, *9*, 2742–2748. [CrossRef]
111. Czibula, G.; Czibula, I.G.; Marian, Z. An effective approach for determining the class integration test order using reinforcement learning. *Appl. Soft Comput.* **2018**, *65*, 517–530. [CrossRef]
112. Kacmajor, M.; Kelleher, J.D. Automatic acquisition of annotated training corpora for test-code generation. *Information* **2019**, *10*, 66. [CrossRef]
113. Song, X.; Wu, Z.; Cao, Y.; Wei, Q. ER-Fuzz: Conditional code removed fuzzing. *KSII Trans. Internet Info. Syst.* **2019**, *13*, 3511–3532. [CrossRef]
114. Rauf, A.; Ramzan, M. Parallel testing and coverage analysis for context-free applications. *Clust. Comput.* **2018**, *21*, 729–739. [CrossRef]
115. Shyamala, C.; Mohana, S.; Gomathi, K. Hybrid deep architecture for software defect prediction with improved feature set. *Multimed. Tools Appl.* **2024**, *83*, 76551–76586. [CrossRef]
116. Bagherzadeh, M.; Kahani, N.; Briand, L. Reinforcement Learning for Test Case Prioritization. *IEEE Trans. Softw. Eng.* **2022**, *48*, 2836–2856. [CrossRef]
117. Tang, Y.; Dai, Q.; Yang, M.; Chen, L.; Du, Y. Software Defect Prediction Ensemble Learning Algorithm Based on 2-Step Sparrow Optimizing Extreme Learning Machine. *Clust. Comput.* **2024**, *27*, 11119–11148. [CrossRef]
118. Xing, Y.; Wang, X.; Shen, Q. Test Case Prioritization Based on Artificial Fish School Algorithm. *Comput. Commun.* **2021**, *180*, 295–302. [CrossRef]
119. Omer, A.; Rathore, S.S.; Kumar, S. ME-SFP: A Mixture-of-Experts-Based Approach for Software Fault Prediction. *IEEE Trans. Reliab.* **2024**, *73*, 710–725. [CrossRef]
120. Shippey, T.; Bowes, D.; Hall, T. Automatically Identifying Code Features for Software Defect Prediction: Using AST N-grams. *Inf. Softw. Technol.* **2019**, *106*, 142–160. [CrossRef]
121. Giray, G.; Bennin, K.E.; Köksal, Ö.; Babur, Ö.; Tekinerdogan, B. On the use of deep learning in software defect prediction. *J. Syst. Softw.* **2023**, *195*, 111537. [CrossRef]
122. Albattah, W.; Alzahrani, M. Software defect prediction based on machine learning and deep learning techniques: An empirical approach. *AI* **2024**, *5*, 1743–1758. [CrossRef]
123. Li, J.; He, P.; Zhu, J.; Lyu, M.R. Software defect prediction via convolutional neural network. In Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS), Prague, Czech Republic, 25–29 July 2017; pp. 318–328. [CrossRef]
124. Afeltra, A.; Cannavale, A.; Pecorelli, F.; Pontillo, V.; Palomba, F. A large-scale empirical investigation into cross-project flaky test prediction. *IEEE Access* **2024**, *12*, 131255–131265. [CrossRef]
125. Begum, M.; Shuvo, M.H.; Ashraf, I.; Al Mamun, A.; Uddin, J.; Samad, M.A. Software defects identification: Results using machine learning and explainable artificial intelligence techniques. *IEEE Access* **2023**, *11*, 132750–132765. [CrossRef]

126. Ramírez, A.; Berrios, M.; Romero, J.R.; Feldt, R. Towards explainable test case prioritisation with learning-to-rank models. In Proceedings of the 2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), Dublin, Ireland, 16–20 April 2023; pp. 66–69. [CrossRef]
127. Mustafa, A.; Wan-Kadir, W.M.N.; Ibrahim, N.; Shah, M.A.; Younas, M.; Khan, A.; Zareei, M.; Alanazi, F. Automated test case generation from requirements: A systematic literature review. *Comput. Mater. Contin.* **2020**, *67*, 1819–1833. [CrossRef]
128. Mongiovì, M.; Fornaia, A.; Tramontana, E. REDUNET: Reducing test suites by integrating set cover and network-based optimization. *Appl. Netw. Sci.* **2020**, *5*, 86. [CrossRef]
129. Saarathy, S.C.P.; Bathrachalam, S.; Rajendran, B.K. Self-healing test automation framework using AI and ML. *Int. J. Strateg. Manag.* **2024**, *3*, 45–77. [CrossRef]
130. Brandt, C.; Ramírez, A. Towards Refined Code Coverage: A New Predictive Problem in Software Testing. In Proceedings of the 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), Napoli, Italy, 31 March–4 April 2025; pp. 613–617. [CrossRef]
131. Zhu, J. Research on software vulnerability detection methods based on deep learning. *J. Comput. Electron. Inf. Manag.* **2024**, *14*, 21–24. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.